Linköping Studies in Science and Technology
Dissertation No. 2460

Improved Tooling for Digital Hardware Development: Spade, Surfer, and more

Frans Skarman

LINKÖPING UNIVERSITY



Linköping studies in science and technology.
Dissertations, No. 2460

Improved Tooling for Digital Hardware Development: Spade, Surfer, and more

Frans Skarman

LINKÖPING UNIVERSITY

Linköping studies in science and technology.
Dissertations, No. 2460

Improved Tooling for Digital Hardware Development: Spade, Surfer, and more

Frans Skarman

frans.skarman@liu.se
Division of Electronics and Computer Engineering
Department of Electrical Engineering
Linköping University
SE-581 83 Linköping
Sweden

ISBN 978-91-8118-176-0 (Print)
ISBN 978-91-8118-177-7 (PDF)
https://doi.org/10.3384/9789181181777
ISSN 0345-7524

© 2025 Frans Skarman and Licensed under CC BY-NC, unless otherwise stated.
https://creativecommons.org/licenses/by-nc-nd/4.0/

Printed by LiU-Tryck, Linköping, Sweden 2025

Till mamma, pappa och Björn

Abstract

Hardware complexity is ever-growing but the front-end tools used to design hardware are not keeping up, especially when compared with software tooling. While software and hardware have fundamental differences, there is enough overlap between the domains to warrant taking ideas and inspiration from software tooling to build better hardware tooling. This dissertation is made up of three parts, each focused around a tool that was built in part based on this idea.

The first tool is Spade, a hardware description language built with the explicit goal of improving developer productivity by taking inspiration from software programming languages. Some features are carried over outright: the type system, a build system with easy dependency management, and a compiler that produces helpful error messages. Most features however, are built specifically for hardware based on design philosophy from software, these include an abstraction for correct by construction pipelining, linear types for modeling memory ports, and the ability to define new custom hardware-centric abstractions such as ready valid interfaces.

The second tool is Surfer, a waveform viewer built from the ground up to accommodate new hardware design workflows. One example of this is integration with modern HDLs like Spade to allow full use of their complex type systems. Other examples include the ability to embed the waveform viewer in bigger project, to control it externally, and to use it in web technologies. The viewer has already seen widespread adoption, has integration with Chisel, Clash, and RHDL in addition to Spade, and is used as a component of several proprietary and open source tools.

The final part of the thesis is centered around Cinnabar, a new high level synthesis tool. It lays the groundwork for a way for domain experts to work with hardware engineers to build accelerators for model based control applications. To do this efficiently, it is helpful if the domain experts can work on the modeling largely independently of the hardware engineers working on the hardware architecture. This is achieved by a high level synthesis tool that compiles a simulation model to *efficient* hardware without requiring any domain expertise. In parallel, a hardware engineer can develop the hardware which executes the simulation, a task which requires less domain expertise and where high level synthesis typically performs worse than a handwritten solution.

In order to tie these works together, a simulation model of a hybrid electric vehicle synthesized with high level synthesis was combined with hardware written in Spade for performing dynamic programming. Together, this allows real time use of an algorithm for optimizing fuel efficiency, which has traditionally only been possible off-line when executing on CPUs.

Populärvetenskaplig Sammanfattning

Komplexiteten för hårdvara har ökat rejält sedan 90-talet. Då bestod de största chipen av lite över 1 miljon transistorer per chip, medan moderna chip består av långt över 10 miljarder transistorer. Verktygen som används för att designa dessa chip har dock utvecklats betydligt långsammare, speciellt de programmeringsspråk och omkringligande verktyg som används för att beskriva hårdvarans beteende. De vanligaste hårdvarubeskrivande språken designades under mitten på 80-talet och har bara blivit inkrementellt uppdaterade sedan dess. Den här avhandlingen består av tre delar som är centrerade runt var sitt projekt som moderniserar hårdvaruutveckling.

Den första delen handlar om Spade, ett hårdvarubeskrivande språk som tar inspiration från moderna mjukvruspråk. Komplexiteten för mjukvara har också ökat kraftigt sedan 90-talet, men där har även programmeringsspråken följt med i utvecklingen, vilket lett till både ökad produktivitet och reduktion av antal fel. Spade är utvecklat för att ge liknande fördelar i hårdvaruutveckling. I vissa fall, till exempel dess typsystem, går det att direkt ta funktionalitet från mjukvruspråk, medan det i de flesta andra fall krävs nya abstraktioner som låter utvecklare resonera om sin design på en högre nivå, utan att tappa kontrollen över viktiga detaljer.

För att nå nya prestandamål när Moores lag stannat av krävs dedikerad hårdvara gjord specifikt för varje applikation. Att utveckla det krävs att hårdvaruutvecklare arbetar tillsammans med domänexperter. Högnivåsyntes marknadsförs ofta som ett verktyg som kan lösa detta genom att låta domänexperter skriva vanlig mjukvarukod som automatiskt transformeras till effektiv hårdvara. I praktiken krävs dock hårdvaruexpertis för att använda dessa verktyg effektivt. Vi föreslår ett alternativt arbetsflöde för hårdvara som används för simulering och optimering. Domänexperter använder högnivåsyntes, för att transformera simuleringsmodeller till hårdvara. Det är oftast möjligt att göra utan hårdvaruexpertis utan att offra för mycket presetanda vilket låter domänexperter iterera modellen utan att tänka på resulterande hårdvaran. Parallellt utvecklar en hårdvaruutvecklare kontrollogik i till exempel Spade för att exekvera modellen. Här krävs mindre domänexpertis och effektiv hårdvara är viktigare. Med det i åtanke har vi utvecklat Cinnabar, ett högnivåsyntessystem utvecklat för simuleringsmodeller. Cinnabar kan göra optimeringar som generella högnivåsyntessystem inte kan göra eftersom simuleringsmodeller ofta är toleranta mot små numeriska fel. Med detta arbetsflöde kan vi bygga effektiv hårdvara som kan minska bränsleförbrukningen i hybridbilar.

Ett oundvikligt steg i hårdvaruutveckling är felsökning. När hårdvara simuleras sparas värden på alla signaler över tid, och dessa signaler kan inspekteras i efterhand under felsökning. Det görs oftast med en så kallad vågformsvisare. Det tredje projektet är Surfer, en ny vågformsvisare som är designad för nya moderna arbetsflöden. Den integrerar med moderna hårdvarubeskrivande språk, bland annat Spade, för att visa komplexa typer och går att utöka för att enkelt lägga till ny funktionalitet. Projektet har fått stor spridning, speciellt innom hårdvara med öppen källkod och det har integrerats med flera hårdvaruutvecklingsverktyg, både med öppen källkod och proprietära verktyg.

Acknowledgements

The past 6 years have been incredible, far exceeding my expectations for what a PhD can be. This is in no small part thanks to the amazing people that have been there along the way.

First and foremost is of course my advisor Oscar. Thank you for taking me on as a PhD student and showing me the ropes of research. Thank you especially for allowing me, no, *encouraging me* to pursue the random side projects I started when they showed signs of promise. Nearly every PhD student will say that the most important part when picking a PhD position is to pick an advisor you will like working with. I can only echo that sentiment, working with you has made the past six years a breeze.

I have also had two amazing co-supervisors. Daniel, working with you on the vehicular systems research was an absolute delight, and even when I moved to even more fun research you have provided great support. And Mattias, while we did not do much direct research together, I have always been able to count on you for any questions and extremely valuable feedback on my papers.

Mikael, sharing an office with you has been one of the best parts of this process, and I look forward to a long continued friendship with a steady stream of top tier memes to come. Despite how insane we have both felt at times, you have been an important part in me staying sane.

To my fellow PhD students, Olle, Theodor, Linus, and Cheolyong, thank you for ~~the well executed raids on VS~~ your company on our ever important coffee breaks. Having great friends to rant with about work and non-work alike has been invaluable.

To the rest of my colleagues at DA, thank you for all you have done for me over the past years. Thank you especially for the quality of your teaching, the courses I took with you during my bachelor were some of the best, and getting to do my teaching duties with such a top tier group of teachers has been incredibly rewarding. And speaking of colleagues, thank you especially Anders and Kent for the ever interesting lunch discussions at Valla.

To Emil and Malcolm, thank you for invaluable friendship over the past 11 years. Hopefully we will still be friends when we get shocked about it being 11 years since it was 11 years since we started at university. Thank you, Jakob, for being my life long friend all the way from pre-school.

To everyone in LiTHe Kod, past and present, your friendship has been incredible and I would not be where I am today without having a community of like minded people who are as passionate about the intricacies of programming as I am. A few people here deserve a mention by name: Gustav, your friendship means a lot to me, and your contributions to Spade, especially the tooling you built, are essential for the language being where it is today. Astrid, hacking on weird projects with you has been amazing, and your insights on language design have made many of the recently added Spade features miles better than they would have been without you.

To my friends in “unsafe_unsafe”, your friendship means at least as much to me as my real world friends. You were also the first people I told about Spade,

and your encouragement and help along the way has been instrumental to its success. Cuddlefishie, you deserve a special mention, thank you for encouraging me to build a community around the language, and of course for teaching me about Spade-fishes.

The dream of any open-source maintainer is for other people to not only use your software, but to contribute to it. I have been lucky enough for that to happen twice. To everyone who has contributed to Spade, thank you! Thank you especially to Ethan and "DasLixou", who have helped design new language features, encouraged me to stream the development, and taken on the task of building some of the tools that a language means but I have not had the time to build.

Surfer has seen even more contributors and I am grateful to each and every one of you! Thank you especially to Kevin for the Wellen library which has enabled more cool features than I could have dreamt of. Thank you as well to Yehowshua for the work you did on the FastWave without which I would never have started the project.

A few fellow scientists also deserve special mention. Thank you Lucas for great collaboration and even better friendship. Thank you Julian for bringing so many neo-HDL authors together in "hdl-other" Thank you Oron for inviting me to talk about Spade at the FCCM workshop. The legitimacy that gave to the project is what moved it from hobby project to main research project.

With the acknowledgements getting close to two pages, I am literally running out of space. To everyone I have missed that has been there along the way, thank you!

I have of course saved the most important for last. My parents Eva and Erik. For everyone else I have had something concrete to say, but for you I cannot come up with anything sufficient. Instead, I will just say thank you for *everything*! A man cannot dream of better parents than you.

Linköping, May 2025
Frans Skarman

Contents

I Background

1 Introduction **2**
- 1.1 Thesis Contributions ... 3
  - 1.1.1 Spade ... 4
  - 1.1.2 Cinnabar and Hybrid Electric Vehicle Optimization ... 6
- 1.2 Structure and Included Publications ... 8

2 The Hardware Design Process **12**
- 2.1 Specification ... 13
- 2.2 Design Entry and Abstraction ... 14
- 2.3 Lowering ... 17
  - 2.3.1 Synthesis ... 17
  - 2.3.2 Place and Route ... 18
  - 2.3.3 Post Place and Route Analysis ... 18
- 2.4 Validation and Verification ... 18

3 Hardware Description Languages **19**
- 3.1 A Taxonomy ... 21
  - 3.1.1 Embedded and Standalone HDLs ... 21
  - 3.1.2 Abstraction Level ... 22
  - 3.1.3 Type Systems ... 23
- 3.2 Existing HDLs ... 24
  - 3.2.1 Chisel, SpinalHDL, and Amaranth ... 26
  - 3.2.2 Clash ... 27
  - 3.2.3 Filament ... 27
  - 3.2.4 DFHDL ... 27
  - 3.2.5 BlueSpec ... 27
  - 3.2.6 Verification Focused Languages ... 28
- 3.3 Contributions ... 28
  - 3.3.1 Pipelining ... 28
  - 3.3.2 Type System ... 29
  - 3.3.3 Tooling ... 29
  - 3.3.4 Effects on the HDL Ecosystem ... 30

4 High-Level Synthesis **30**
- 4.1 Typical Static HLS Flow ... 32
- 4.2 Dynamic HLS ... 33
- 4.3 Accelerator Design Languages ... 33
- 4.4 Contributions ... 33

5 Analyzing Simulation Results **34**
- 5.1 Contributions ... 36

6 Dynamic Programming for Optimal Control **37**
6.1 Hybrid Electric Vehicles ... 39
6.2 Optimal Control ... 40
6.3 Dynamic Programming ... 41
6.4 Hardware for Dynamic Programming ... 42
6.5 Contributions ... 43

7 Conclusions and Future Work **44**
7.1 Future work ... 45
7.1.1 Spade ... 46
7.1.2 Surfer ... 47
7.1.3 Cinnabar and Hybrid Electric Vehicles ... 47

Bibliography **48**

II Publications

A Spade – A Modern Hardware Description Language **59**
1 Introduction ... 61
2 Motivation for a Software-Inspired HDL ... 62
2.1 Technical Benefits and Abstraction ... 63
2.2 Performance ... 64
2.3 Tooling ... 64
2.4 Hardware is Not Software ... 64
3 The Spade Language ... 65
4 Pipelines ... 66
4.1 Stage References ... 67
4.2 Dynamic Behavior ... 68
5 Type System ... 69
5.1 Sum Types and Pattern Matching ... 71
5.2 Generics ... 73
5.3 The Option Type ... 73
5.4 Methods ... 74
5.5 Traits and Requirements ... 75
5.6 Higher Order Circuits ... 75
5.7 Linear Types and Ports ... 76
5.8 Latency-Insensitive Combinators ... 78
6 Tooling ... 79
6.1 Compiler Errors ... 79
6.2 Swim - The Spade Build Tool ... 80
6.3 Testing ... 81
6.4 Waveform Analysis Tools ... 82
6.5 Surrounding infrastructure ... 82
7 Implementation ... 82
8 Related Work ... 83
9 Conclusion ... 84
Bibliography ... 84

B A Case for Option Types in Hardware Design **89**
1 Introduction ... 91
2 The Option Type ... 92
3 Brief Introduction to Spade ... 94
4 Enforcing Validity Checks ... 95
5 Combinators ... 96
6 Latency Insensitive Interfaces ... 97
7 Synthesizing Option Types ... 99
8 Optimizations ... 100
8.1 Sparse Dot Product ... 102
8.2 Ethernet FIR Filter ... 104
8.3 Enabling Performance Tuning ... 105
8.4 FPGA Considerations ... 105
9 Generalizing to Sum Types ... 106
10 Conclusions ... 106
Bibliography ... 107

C Enhancing Compiler-Driven HDL Design with Automatic Waveform Analysis **109**
1 Introduction ... 111
2 Motivating Example ... 113
3 Languages ... 116
3.1 Spade ... 116
3.2 Waveform Analysis Language (WAL) ... 118
4 Using and Defining Analysis Passes ... 120
4.1 Struct Annotation ... 120
4.2 WAL Code ... 121
4.3 Making the Pass Discoverable ... 121
5 Finite State Machine Analysis Pass ... 123
6 Implementation ... 124
7 Related Work ... 126
8 Conclusion ... 126
9 Acknowledgments ... 127
Bibliography ... 127

D Surfer – An Extensible Waveform Viewer **129**
1 Introduction ... 131
2 Extensible Translator System ... 132
2.1 Hardware Description Language Support ... 132
2.2 Instruction decoding ... 133
2.3 Project-Specific Python Translators ... 134
3 Integrating Surfer with the EDA Ecosystem ... 134
3.1 Embedding Surfer in Web Applications and Visual Studio Code ... 135
3.2 Controlling Surfer from a Third Party Tool ... 135
3.3 Interactive Simulation ... 136

3.4 Tools Integrating Surfer ... 137
4 A Performant Waveform Backend ... 138
4.1 Remote Servers and Continuous Integration ... 139
5 Software Project ... 139
6 Conclusion ... 140
Bibliography ... 140

E A Tool to Enable FPGA-Accelerated Dynamic Programming for Energy Management of Hybrid Electric Vehicles 143
1 Introduction ... 145
2 Problem statement ... 146
3 Powertrain Energy Management using Dynamic Programming ... 147
3.1 Dynamic Programming ... 147
4 FPGA ... 148
4.1 Related Work ... 149
5 Vehicle Model ... 149
6 Vehicle Model to FPGA Conversion ... 151
6.1 Cinnabar Internals ... 152
6.2 Executing the Vehicle Model ... 153
7 Performance Evaluation ... 153
7.1 Output Quality ... 154
7.2 Runtime Performance ... 155
8 Conclusions and Future Work ... 157
Bibliography ... 157

F Leveraging LLVM to Build Fully Parallel Pipelined FPGA Hardware 159
1 Introduction ... 161
2 Preliminaries ... 162
2.1 High-Level Synthesis ... 162
2.2 LLVM Intermediate Representation ... 163
2.3 Output Hardware Architecture ... 164
2.4 Instruction Conversion ... 165
3 Handling Control Flow ... 166
3.1 A Mask Based Algorithm ... 167
3.1.1 Shortcomings of the Mask Method ... 168
3.2 A Path-Based Algorithm ... 168
3.3 Conditions From Paths ... 169
3.3.1 The `find_paths` Algorithm ... 170
3.3.2 Some Notes About Efficiency and Performance ... 172
3.3.3 Caching Conditions ... 172
3.4 Undefined Branches ... 173
3.5 A Note on Calling Conventions ... 174
4 Performance Evaluation ... 174
4.1 Synthetic Programs ... 174
4.2 Real World Kernels ... 175

4.2.1 HEV Model 175
4.2.2 Mandelbrot 175
4.2.3 Pneumatic valve 176
4.3 Compiler Options 176
4.4 Results 177
5 Discussion 182
6 Conclusions 182
Bibliography 183

G Scalable FPGA Implementation of Dynamic Programming for Optimal Control of Hybrid Electrical Vehicles 185
1 Introduction 187
2 Dynamic Programming 188
2.1 The Dynamic Programming Algorithm 189
2.2 Next State and Input Cost 189
3 Related Work 190
4 Proposed Architecture 190
4.1 Functional Unit Structure 191
5 Performance Characteristics 194
6 Synthesis Results 196
7 Conclusions 197
Bibliography 197

Part 1
Background

Chapter 1
Introduction

Turing Award winners John Hennessy and David Patterson recently [1, 2] argued that we are at the cusp of another golden age of computer architecture where cost, performance, energy usage, and security of computers will improve significantly. They argue that part of these improvements will come from a shift from general purpose CPUs to domain-specific hardware and heterogeneous processors, but that achieving this requires major breakthroughs in design time and cost. Hennessy and Patterson further argue that a goal is for *small* teams to be able to design chips tailored for their application, and that this requires hardware design to become much more efficient, and *more like modern software design* [1]. This final part, becoming more like software design is a central theme of this thesis which revolves around building new software-inspired tools for designing hardware.

Tooling is a major part of what makes modern software design so efficient. For example, developments in language design have enabled significant boosts in developer productivity with Google claiming a 2× improvement in developer productivity with Rust and Go over C++ [3]. In a similar vein, R. K. Fulton et al. [4] conducted a survey which highlights several language and ecosystem features that contribute to developers feeling more productive in Rust than in other languages. Beyond productivity improvements, several studies have shown significant gains in program correctness [4, 5, 6, 7] through adopting modern programming languages. Being able to eliminate classes of bugs outright should be an especially tempting proposition in hardware where significant engineering effort is spent doing verification. Type systems can be a great aid in finding bugs, and it is sometimes claimed that "type systems are the parts of formal methods that we figured out how to make easy" [8]. While one may be tempted to think that only improving the productivity of the *easy parts* is an insignificant gain, that is not necessarily the case. Type systems provide very quick feedback to design engineers, much faster than synthesis or simulation runs, and without anyone having to spend time writing formal tests for the tested properties. In addition, verification engineers can focus on proving the non-easy properties that actually require their expertise, instead of also having to prove "trivial" properties that the type system can handle. Finally, in Field Programmable Gate Array (FPGA) projects where formal verification is not as common, a type system is an easy way to guarantee the absence of some bugs which would otherwise need to be found manually through tests.

Another area where software engineering has made great advances is in separation of concerns, something which is especially evident in traditional web

development which generally consists of three parts: content written in HTML, style and design in CSS, and application logic written in JavaScript. With only a loose coupling between these three areas, they can be worked on in parallel by people with different skill sets. Designers can focus on making the site look great, primarily using CSS and without having to know much if any JavaScript or HTML, similarly programmers can use JavaScript to define behavior of the website without having to concern themselves with the visual appearance of the interface. Once styling and behavior is done, content can be adjusted in HTML by people without JavaScript or CSS experience. Admittedly, with the emergence of JavaScript libraries such as React, Vue, and Svelte, more things have moved into JavaScript or TypeScript, but the separation of concerns still exists. In hardware, being able to split responsibilities like this is rarer. High-Level Synthesis promises to allow non-domain experts to write hardware themselves, but in practice hardware expertise is still required to get good results, forcing domain experts to either acquire those skills, or to work closely with hardware experts to design circuits. This too is an area where learning from software would bring us closer to Henessy and Patterson's goal of small teams being able to design chips *tailored for their application.*

Taking a step back and looking at the state of hardware design from afar, software and hardware development may look very similar to each other. While the resulting artifact is quite different, the job of a hardware design engineer and a software developer is both to write *code* that describes their "desired artifact" in a programming language, and through a sequence of tools, that code is transformed into a program or chip that (hopefully) behaves as the programmer expected.

With this in mind, it may seem strange that Hennessy and Patterson have to argue that hardware design should be more like software design. However, despite the similarities, there is surprisingly little sharing of ideas between software and hardware engineering. An interesting case study for this is the design of the programming languages used in each field. The development in programming language design that has led to such huge boosts in developer productivity in software has not been matched in the hardware domain. Over 90% of developers use Verilog, SystemVerilog, or VHDL [9] which, despite having seen incremental improvements since their inception, are firmly rooted in programming language design from the 1980s. In contrast, of the 18 software languages in use by more than 5% of professional developers, 78% originate in the 1990s or later. For reference, Rust and Go which Google claimed significantly boosts the productivity of their developers over C++ [3] was only released in 2015 and 2009 respectively. This difference is visualized in Figure 1 where software [10] and hardware languages used in FPGA designs [9] are positioned based on the time at which they appeared, and scaled by the percentage of developers using them. From this, one can see that the hardware languages in use today mostly pre-date the software languages by quite a wide margin. Naturally, such a comparison is not without its flaws, languages evolve over time, their creation date does not fully reflect their capabilities, and language adoption can be driven by more than raw developer productivity, even by unrelated factors such as marketing or hype. Instead, this comparison should be seen as an indication of the differences between the ecosystems, rather than a black and white comparison between them.

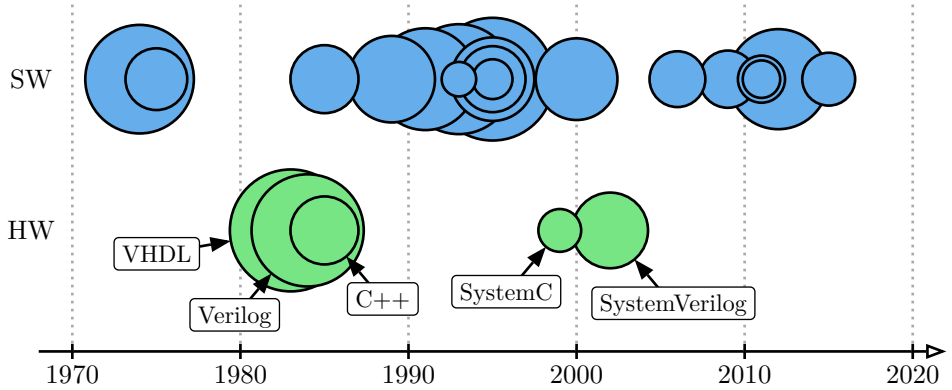


Figure 1: Plot showing the relative number of users and year at which programming languages appeared in software [10] versus hardware [9]. The circles are centered at the year they appeared, and their area corresponds to the percentage of developers who use them.

The comparison between languages is convenient, because there is easily accessible data that indicates a difference. Similar differences exist in other areas as well, though those differences are not as easy to quantify. For example, hardware designs are still largely based on proprietary, often in-house, components while most modern software is fundamentally built on a large ecosystem of open-source libraries that are often maintained and developed by several stakeholders, both companies and individuals. Development tooling is in a similar boat, where compilers and developer environments in hardware are proprietary and developed by large corporations and which are developed very slowly. For example, many commercial synthesis tools and simulators still do not support all of VHDL 2008 16 years after its ratification. In software on the other hand, several compilers support all of C++20 and large sections of C++23 [11]. Both of these issues are reminiscent of the software world in the 1990s before Clang and GCC sped up feature adoption in C++, and before tools like NPM and PIP allowed easy use of the open-source ecosystem.

1.1 Thesis Contributions

The overarching goal of this thesis is to attempt to address some of these problems and bring hardware design a bit closer to that of modern software development, with the focus being on frontend tools like programming languages and debugging tools rather than synthesis or place-and-route tools. Broadly, the thesis consists of two separate but related tracks. The first is centered around a new Hardware Description Language (HDL) called Spade which draws inspiration from modern software languages. The second track is centered around accelerating dynamic programming to optimize the fuel efficiency of Hybrid Electric Vehicle (HEV). The main contribution here is Cinnabar, a High-Level Synthesis (HLS) tool built for synthesizing simulation models written in C++ into hardware. Another contribution is a methodology for hardware engineers and domain experts to work

together on building a complete system for this kind of model-based optimal control application.

1.1.1 Spade

As mentioned earlier, the software world has made great progress in developer productivity and program correctness through developments in programming languages. Spade is an attempt to bring these ideas into the hardware world in order to take advantage of these developments. While some innovations such as the use of strong type systems to prevent bugs can be ported directly from software to hardware, the two domains have some fundamental differences and sometimes require a different approach. Here, Spade instead takes the approach of borrowing design philosophy rather than language features.

One important area where this has been applied is in the design of abstractions where successful programming languages identify problematic areas in the development cycle and attempt to build abstractions that prevent these problems. An excellent example of this is the memory safety that Rust provides, which fundamentally is an abstraction around the memory that prevents misuse of memory resources. By shifting the reasoning from allocations and pointers to ownership and borrowing, developers can, together with the compiler, accurately reason about the lifetime of memory resources and prevent all use after free and concurrency bugs. This particular problem is at a higher level of abstraction than hardware designs so the key to achieving similar gains in hardware is not to copy the implementation, but the philosophy of building abstractions that categorically prevent bugs by lifting reasoning from a low to a higher level.

With this in mind, Spade includes several abstractions that are specific to hardware. First and foremost is an abstraction for pipelines which statically prevents the use of values which are out of sync or which are not ready, allows easy composition of pipelines, and allows trivial re-timing of circuits. For pipelines with feedback and bypasses, some of these guarantees are reduced, but the language still helps the user reason in terms of pipeline stages instead of individual registers.

The Spade type system also allows defining *new* abstractions without direct support from the compiler itself. One example of this is ergonomic definitions of modules with ready-valid handshaking. Using a library defined in Spade, users can easily compose modules using these interfaces together to perform computations as fast as the slowest component allows and with syntax that makes it easy to get an overview of the chain of computation.

Additional abstractions include memories and ports which are backed by linear types. This allows units like memories to return ports as single structures which can be passed as a whole to consumers of the port like accelerators and processors. A linear type system ensures that these ports are used correctly, i.e. they are driven by exactly one driver.

The language will be discussed in much more detail later in the thesis, but as a teaser, Figure 2a contains some Spade code that implements the top module of a CSI2 camera protocol decoder. The `reg` keyword is used in pipelines to split the computation into several pipeline stages. A schematic of the resulting hardware is shown in Figure 2b.

```
pipeline(7) csi2(
  clk: clock,
  rst: bool,
  lanes: [uint<8>; 2]
)
  -> (ShortPackets, Pixels)
{
    let aligned = [
      inst aligner(clk, rst, lanes[0]),
      inst aligner(clk, rst, lanes[1]),
    ];

  reg;
    let merged = merger(aligned);

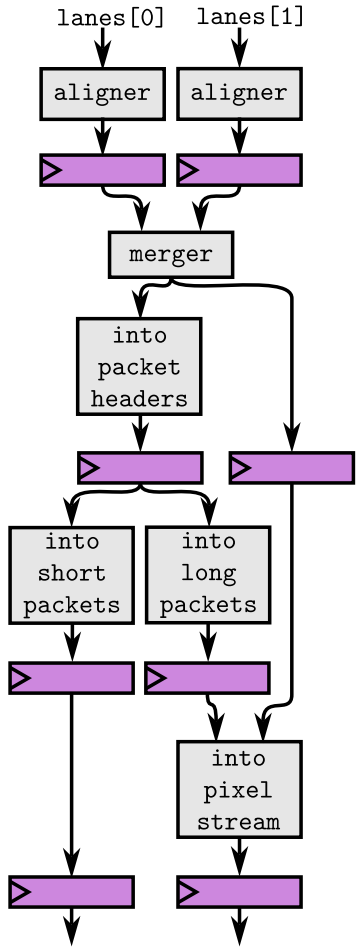
    let headers = merged
        .inst into_packet_headers(clk, rst);

  reg;
    let short_packets = headers
        .into_short_packets();
    let pixel_headers = headers
        .into_long_packets();

  reg;
    let raw_pixels = pixel_headers
      .inst into_pixel_stream(
        clk,
        rst,
        merged
      );

   reg;
     (short_packets, raw_pixels)
}
```

(a)



(b)

Figure 2: Spade code and resulting hardware of a top module that decodes the CSI2 camera protocol.

Like modern software languages, Spade emphasizes good tooling that helps developers work faster and more efficiently. This includes a compiler built from the ground up to provide helpful error messages that helps the programmer quickly fix issues. Together with that, the language is designed to prefer reporting errors for ambiguous statements rather than assuming one possible interpretation. For example, operations like truncation that have the potential to remove information are explicit, rather than being inserted implicitly by the compiler. Additional tools include editor integration through tree-sitter and the language server protocol, and a build system that includes dependency management which allowing easy access to an ecosystem of open source (and closed source) libraries (IP blocks).

Since debugging and verification is a time-consuming aspect of most hardware projects, several tools have been built or integrated with Spade to facilitate faster debugging. First, Spade was integrated with the WAL language [12], a program-

ming language for analyzing waveforms after simulation. This integration allows developers to easily programmatically and retroactively analyze simulation results to extract information that would otherwise need to be found visually through waveform viewers, or via more complex test benches. Through integration of WAL with Spade and its build tool, developers are able to distribute not only an IP block of their design, but also include scripts that analyze the performance or finds issues when the IP block is used. For example, authors of an interconnect library like Wishbone or AXI can include a script that visualizes the utilization over time of the bus. By bundling this with the library any user can, with a single command, get insights about the performance of their design that would otherwise require significant effort to extract.

Finally, with an HDL that makes heavy use of complex types, it is very difficult to debug a circuit from just looking at binary, numeric, or hexadecimal representations of signals that conventional waveform viewers usually provide. In order to aid in debugging, a new waveform viewer, Surfer, was developed for Spade which allows more complex visualization of signal values than existing waveform viewers. In the spirit of building a rich open source ecosystem of design automation tooling, Surfer was built to be extensible to support custom visualization, and to be easy to integrate in other projects. A little over two years into its development this has already resulted in several new external integrations including

- Visualization of Chisel types
- Translation of RISC-V instructions and user defined instruction sets
- Visualization of transactions stored in FTR format

It is being used both as a standalone program and is being integrated in several commercial and open source flows.

1.1.2 Cinnabar and Hybrid Electric Vehicle Optimization

A very common use case for FPGAs is to accelerate specific computations to run them faster or in a more power efficient way. One such problem is finding a fuel efficient way to drive hybrid electric vehicles where the vehicle can store excess energy in the battery. If the route ahead is known, one can compute when to store some extra energy, for example running the combustion engine a bit more before when approaching a steep hill where it is known that the combustion engine will struggle. This optimization can be done using dynamic programming, but in order to produce the best results, the algorithm should be able to take into account traffic conditions, and to do so it must be run in real time, something that is not feasible on CPUs. The algorithm used to perform this optimization consists of a simulation model of the vehicle which takes the current state of the vehicle, the current driving conditions, and a set of inputs and produces a new vehicle state along with the *cost* of applying the input. This vehicle model can then be used with dynamic programming to simulate a large number of inputs to the vehicle in order to determine an “optimal” sequence of inputs that reaches the destination with the best fuel efficiency.

Like many accelerator projects, implementing this requires both hardware expertise to build efficient hardware, and domain expertise to design the vehicle

model and design the algorithms to be accelerated. As mentioned in the introduction, in the software world, particularly in web development, there are often ways for different domain experts to work together on a shared project, with minimal requirements on understanding "the other side", but this is more limited in the hardware space. HLS is often marketed as allowing software developers or domain experts to go from zero to working hardware without any hardware experience. For current HLS tools however, this is often not the case, instead hardware engineers end up trying to coerce the HLS tool to give them the hardware with adequate performance.

Here we approach the problem in a slightly different way. First, we use HLS to build hardware for the vehicle simulation model. Since it is a pure function from input to output, it can be pipelined almost arbitrarily and a high throughput can be achieved with minimal input from a hardware domain expert. This allows the vehicle simulation model to be designed in C++ which the vehicle domain expert is more likely to be familiar with. While the vehicle domain experts are iterating on the model, hardware experts can concurrently build the hardware that drives the optimization process where less domain knowledge is required, and where more tricky problems such as memory allocation and scheduling can be done by the hardware engineer in an HDL built for the task, rather than by coercing a HLS tool into a somewhat efficient design using pragmas.

As part of this, we also propose a new HLS tool, *Cinnabar*, which is built specifically for these high throughput pipelined models. By having a high level synthesis tool that only focuses on building pipelined simulation models, the opportunity to perform interesting optimizations also arises. In particular, since the model is often a simplification of the system being modeled, small errors can be tolerated if they reduce the hardware resources required for the implementation. This allows converting the model to fixed-point formats with smart selection of bit widths that reduces model size while keeping the error compared to the floating-point model within tolerable bounds. Cinnabar therefore does all computations using fixed-point and automatically selects the smallest integer word length required for each variable to guarantee that no overflow occurs for the possible inputs to the model. Ideally, a similar static analysis would be performed to select the number of fractional bits, but as this is a much harder problem to solve, Cinnabar resorts to simulation of the model at different word lengths. This allows the vehicle domain expert to choose a configuration that results in a good tradeoff between accuracy and resource usage.

As a case study for these ideas, a mild hybrid electric vehicle model was synthesized and combined with a handwritten executor for dynamic programming. This system allows the HEV optimization to be performed in real time in a vehicle, something that is only possible to do off-line with execution on a CPU. In addition, this architecture is scalable allowing the developers to select a configuration that achieves an appropriate trade-off between hardware resource usage and computation time.

1.2 Structure and Included Publications

The rest of the thesis follows a familiar structure consisting of two parts. The first part provides an overview of the research field including background and related research, as well as listing the main contributions of the thesis. The second part is made up of published and in-review papers which make up the bulk of the scientific contribution.

The background part is structured as follows: first, Chapter 2 describes how hardware is developed and which tools are involved in going from a specification to working hardware. Chapter 3 explores the state of the art of hardware description languages, and defines a taxonomy that helps categorize the large number of HDL projects that are out there. Chapter 4 discusses the state of the art of HLS and introduces some different approaches to HLS. Next, Chapter 5 discusses debugging tools and approaches that are used for verification of digital hardware. The final chapter of the background, Chapter 6, discusses dynamic programming, especially in the context of hardware acceleration and its use for optimal control.

The rest of the thesis consists of the papers that form the bulk of the contributions of the thesis and is split into two tracks as described before.

Track 1

The first track is centered around Spade and the projects that sprung from it including the Surfer waveform viewer and integration between Spade and the Waveform Analysis Language (WAL).

Paper A: Spade – A Modern Hardware Description Language

Authors: Frans Skarman, Gustav Sörnäs, and Oscar Gustafsson

Manuscript

Summary: In this paper we provide a detailed description of the Spade HDL including all features that make it stand out from other languages. This includes pipelining, the powerful type system, and surrounding tooling. We also motivate taking features from software languages and adapting them to hardware.

Contributions: FS has done the bulk of the design of the language and written most of the compiler as well as the initial build tool. GS has contributed to the language design and has added most advanced features to the build tool. OG has made some contributions to the compiler and provided feedback throughout the design process. In addition, the Spade compiler and language, being an open source project, has seen several contributions from external contributors [1] [2].

[1] https://gitlab.com/spade-lang/spade/-/graphs/main
[2] https://gitlab.com/spade-lang/swim/-/graphs/main

The paper is an extension of three previously published papers:

> **F. Skarman** and O. Gustafsson, "Spade: An HDL inspired by modern software languages," in *International Conference on Field Programmable Logic and Applications (FPL)*, IEEE, Aug. 2022, pp. 454–455.
>
> **F. Skarman** and O. Gustafsson, "Abstraction in the Spade hardware description language," in *Languages and Tools for Accelerator Design (LATTE)*, Mar. 2023.
>
> **F. Skarman** and O. Gustafsson, "Spade: An expression-based HDL with pipelines," in *Workshop on Open-Source Design Automation (OSDA)*, 31 Mar. 2023, pp. 7–12.

Paper B: A Case for Option Types in Hardware Design

Authors: Frans Skarman and Oscar Gustafsson

Manuscript

Summary: A key feature of Spade is its sum types which can be used to define an option type. Option types are used frequently in modern software languages to ensure correctness and to enable optimization. Most hardware description languages lack option types however, and those that have them are not making full use of their optimization potential. In this work, we argue for using option types for correctness, and show how they can enable new optimizations that a non-type aware synthesis tool cannot do, and which are tedious for a user to do. This is demonstrated with an automatic clock gating optimization pass that can significantly reduce the dynamic power consumption of circuits with very little overhead.

Contributions: FS did the implementation of the option type in Spade and wrote several projects which demonstrate their use to ensure correctness and for building new abstractions in an HDL. OG came up with the initial idea of using option types to do clock gating, and both authors worked out the details on how to do it together. The implementation of the optimization as well as the development of the test cases which demonstrate the effectiveness of the optimization was done by FS.

Paper C: Enhancing Compiler-Driven HDL Design with Automatic Waveform Analysis

Authors: Frans Skarman, Lucas Klemmer, Oscar Gustafsson, and Daniel Große

Published in: Forum on Specification and Design Languages (FDL), 13-15 Sept. 2023

Summary: Debugging and verification is an inevitable step in any hardware design process and effective debugging requires efficient ways to extract information about the system. The Waveform Analysis Language (WAL) [12] enables programmatic

extraction of information from hardware simulation traces. When combined with Spade and its build system, analysis programs can be bundled with libraries to allow the use of complex analysis for things like buses with near zero effort from the user and which is demonstrated with several case studies. The techniques proposed are not only possible with WAL and Spade but are applicable to any HDL and waveform analysis tool.

Contributions: The project was a close collaboration between FS and LK where FS did the bulk of the integration on the Spade side while LK did the same on the WAL side. The paper was primarily written by FS and LK with significant help from OG and DG.

Paper D: Surfer – An Extensible Waveform Viewer

Authors: Frans Skarman, Lucas Klemmer, Daniel Große, Oscar Gustafsson, and Kevin Laeufer

Accepted to: International Conference on Computer Aided Verification (CAV) 2025

Summary: The waveform viewer is the primary way for hardware designers to debug any simulation errors that arise. Open source waveform viewers have seen very little development in recent years, and crucially are hard to extend to support with new features and workflows. The Surfer waveform viewers addresses these shortcomings and enables several features including extensible integration with HDLs such as Spade and Chisel, integration with teaching tools, and the optional client/server architecture that allows debugging of waveforms on simulation servers without downloading the waveform ahead of time. On the technical side, Surfer is built with extensibility in mind, and is built on a new performant and multi-threaded waveform parsing library.

Contributions FS did the initial implementation of the Surfer frontend and designed the translation system that allows the viewer to perform rich translation of bit vectors. The waveform parsing and querying library, *Wellen*, that Surfer now uses as its "backend" was written by KL. OG has made numerous improvements to the project in several areas and has done a lot of the maintenance tasks of reviewing code from external contributors. LK has developed several new features and has done the bulk of the work in making the viewer usable with only keyboard commands as well as developing the waveform control protocol that enables new integration. Surfer has, has seen several contributions from external contributors [3]

[3] https://gitlab.com/surfer-project/surfer/-/graphs/main

Track 2

The next three papers primarily form track 2 and consist of an introduction to the hybrid electric vehicle optimization problem, some technical details about the cinnabar HLS tool, and finally, a full hardware implementation of the optimization that combines the use of HLS for model synthesis with an executor written in Spade.

Paper E: A Tool to Enable FPGA-Accelerated Dynamic Programming for Energy Management of Hybrid Electric Vehicles

Authors: Frans Skarman, Oscar Gustafsson, Daniel Jung, and Mattias Krysander

Published in: IFAC-PapersOnLine, Vol. 53, Pages 15104-15109, 2020

Summary: In this work, we describe the use of FPGAs to accelerate dynamic programming for hybrid electric vehicle fuel efficiency optimization. We go into detail on the model of the vehicle being optimized as well as the dynamic programming algorithm that uses the model. We also introduce the Cinnabar HLS tool which is used to automatically convert the vehicle model into FPGA hardware. We show that with a few parallel execution units, real-time use of the optimization is feasible which enables the use of the algorithm in vehicles as opposed to simply running it off-line.

In addition, we evaluate the effects of varying the fractional word length used in the evaluation and show that the word length can be tuned to trade output accuracy for FPGA resource usage.

Contributions: FS developed the Cinnabar tool, adapted the vehicle model to the tool, and used it to perform the experiments shown in the paper. The vehicle model was developed by DJ who also contributed significantly to the section of the paper that describes the vehicle model. OG and MK provided feedback during the research and helped finalize the paper.

Paper F: Leveraging LLVM to Build Fully Parallel Pipelined FPGA Hardware

Authors: Frans Skarman and Oscar Gustafsson

Manuscript

Summary: In this work we present several variations of an algorithm that is used in Cinnabar to convert control flow from the sequential LLVM intermediate representation into multiplexers with a minimal expression for the select inputs. This is done through a new algorithm as well as adaptations of some existing algorithms used for auto-vectorization. Benchmarks show that there is no clear advantage

to either algorithm, instead the performance of each varies from benchmark to benchmark. This work was submitted, and the reviewers wanted comparisons with existing algorithms. However, we have found no papers describing how this is solved in other tools which makes such a comparison difficult.

Contributions: FS did the bulk of the implementation of the algorithm and wrote most of the paper, with significant help in both tasks from OG.

Paper G: Scalable FPGA Implementation of Dynamic Programming for Optimal Control of Hybrid Electrical Vehicles

Authors: Frans Skarman and Oscar Gustafsson

Published in: Design and Architectures for Signal and Image Processing (DASIP), Lecture Notes in Computer Science Vol. 14622, pp. 27-39 Jan. 17-19 2024

Summary: In this work, we tie together the two tracks by designing an architecture that can be used to drive the execution of the dynamic programming algorithm for HEV optimization. The control logic is written in Spade and drives a vehicle model produced by Cinnabar or another HLS tool. The architecture is scalable to support a configurable number of execution units which gives designers freedom to trade off performance for resource usage. We show that with this architecture, real time use of the dynamic programming algorithm is possible on FPGAs, and that the speedup of this hardware over CPUs is larger than that of a similar algorithm running on GPUs.

Contributions: FS did the implementation of the algorithm, performed the experiments and wrote the paper, all steps with help from OG. Some initial groundwork for the architecture was done by Isak Paulander as part of an unfinished master thesis.

Papers not included in the thesis

Finally, the following papers were written and published while working on the thesis, but are not included in the thesis.

F. Skarman, O. Gustafsson, D. Jung, and M. Krysander, "Acceleration of simulation models through automatic conversion to FPGA hardware," in *International Conference on Field-Programmable Logic and Applications (FPL)*, Gothenburg, Sweden, 2020, pp. 359–360.

L. Klemmer, **F. Skarman**, O. Gustafsson, and D. Große, "Surfer: A waveform viewer as dynamic as RISC-V," in *RISC-V Summit Europe*, 2024.

Chapter 2
The Hardware Design Process

Hardware design, be it developing a bitstream for an FPGA or a mask for chip manufacture, is a process that involves many steps, both manual and automated. At a high level, this process can be split into four phases: specification, design entry, synthesis, and verification. Figure 3 shows an overview of these phases and the steps that go into each one. The figure has several branches and should be interpreted as a *choose your own adventure*, rather than a comprehensive listing of steps that *have* to be included. For example, most projects are developed manually in an HDL or in a more automated manner through HLSs, not both. The process also naturally includes going back to previous steps to address any issues found in later steps, but these back edges have been left out of the figure for clarity. How much and how often backtracking is done depends on the project, in a waterfall approach, the specification is ideally finalized before any development starts, whereas agile methodologies encourage a much more iterative approach. The figure also highlights the areas where this thesis makes contributions: in the development of a new HDL and its compiler, the development of a HLS tool, as well as debugging tools used to analyze and debug simulation results.

The rest of this chapter goes into more detail about the steps and the tools involved. Readers who are in a hurry and who are already familiar with the hardware design process, specifically the role of HDLs, HLS tools, and verification can skip most of this and jump ahead to Chapter 3, though Section 2.2 is worth reading as it highlights some underlying motivation for all the projects that make up this dissertation.

2.1 Specification

Before any tools are involved, hardware design typically starts off with a specification of some kind that describes what the hardware should do and its structure. The amount of time and effort spent on this, and how in-depth it is, depends heavily on the project and organization. It can range from a small one-person project with only an informal idea of what the project should do, to a large-scale project with hundreds of developers and significant details about the design being decided in the specification phase before a single line of code is written.

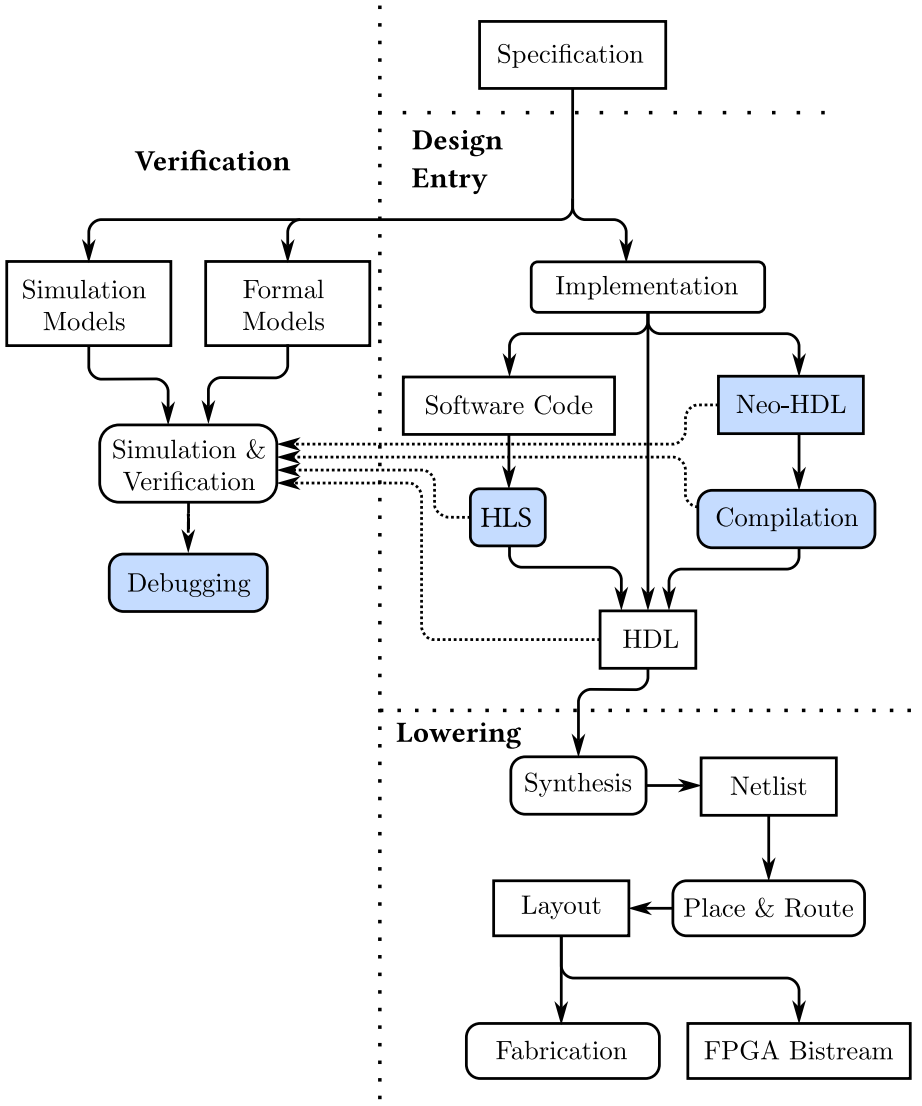


Figure 3: Overview of the hardware design process. The areas primarily addressed in this thesis are highlighted blue.

2.2 Design Entry and Abstraction

Once a specification has been established, the interesting (from the point of this thesis at least) work starts, the exact implementation must be decided, almost always through the use of a programming language in one form or another. An unavoidable decision at this stage is which level of abstraction to use to implement the hardware. It is not uncommon for hardware designers to have a somewhat dogmatic view of abstraction where the assumption is that a higher-level language is necessarily less "performant" than a lower-level language. While there is truth

to this in many cases — current HLS will not give the same performance as hand-written Verilog, there are some underlying assumptions in this general statement that make it somewhat misleading.

Abstraction is one of the primary knobs a language designer can tune when designing their language, as it lets them decide what details are hidden from the user, which details the user *has* to care about, at what level they have to care about them, and how explicit they have to be about those details [13]. To give a few concrete examples of this, at the lowest level of abstraction, a digital circuit can be built by hand by drawing the masks that are used to form the chip. Here, the user has full control, but they also have to worry about physical placement, routing, analog effects, manufacturing details, and so on. This is completely infeasible for even moderately sized digital chips which means that the vast majority of designs are built at a higher-level of abstraction.

The lowest level of abstraction available in Verilog and VHDL is the *netlist* representation. Here, the user can reason in terms of logic gates, groups of logic gates, and their interconnection. This almost completely eliminates the possibility of reasoning about physical placement but since synthesis tools generally do a better job at performing physical placement than any human designer can, at least for a large chip, it is a tradeoff that is almost always made.

The netlist level is also relatively low, and many designers instead work at the Register Transfer Level (RTL) level where circuits are described in terms of how values are transformed from one register to the next. The RTL abstraction eliminates the need to think about individual logic gates, but a moderately experienced designer can still infer roughly what hardware a given RTL description will result in. In that way, RTL raises the abstraction level to allow the designer to express their design in a more easy to reason about way, while still largely being able to affect the exact resource usage of the chip. The analog effects of the underlying chip are still a concern, but to an RTL designer, the exact details are inconsequential; what they care about is ensuring that the chip can run at the target frequency. Here, static timing analysis is used to capture all the analog effects and expose only the details the developer is interested in: how much time does the slowest signal in the chip take to propagate between two registers.

So far, the abstractions discussed have been very advantageous; they hide *unimportant* details such as physical design, which a tool can solve, and they reframe unavoidable details, like analog effects, in a way that captures only the *important* details. However, as the abstraction is lifted even further, tradeoffs usually have to be made.

A large fraction of digital designs are sequential, meaning that the behavior over multiple clock cycles is what a designer is interested in, not just the behavior from one clock cycle to the next. At the RTL level, this is something that a designer *cannot* reason about directly, only indirectly, for example by building state machines. However, as this is a low level of reasoning that requires time and expertise, finding more productive abstractions around cycle-to-cycle timing is a central goal of many hardware design tools, in academia and industry alike.

High-Level Synthesis takes a somewhat extreme approach where software languages are compiled directly to hardware. However, the input languages to HLS

have no way to express precise timing or hardware mapping, and very limited ways to express parallelism explicitly. This means that with HLS the user is left at the mercy of the underlying tool with only limited control over the exact hardware that is generated. In practice, the resulting hardware is often less efficient than what a designer in RTL can achieve which gives the HLS abstraction a "cost".

Between HLS and RTL, there are several different approaches to timing. Some languages like Filament [14], SUS [15], and to some extent Spade, largely retain the RTL level but allow the developers to encode the timing behavior of modules in the language with the compiler reporting errors when the timing requirements between modules is mismatched. This is another use for abstraction: expose some common source of problems explicitly, and abstract it away with a system that guarantees that such problems cannot occur. As a non-hardware example, the ownership system that Rust uses to completely eliminate memory-related bugs reframes the problem of memory allocations and pointers as one of ownership and borrowing.

Yet another use for abstraction is to provide separation of concerns. A hardware designer must reason both about the output of their hardware being correct, and about the implementation details of when and where their computation is performed. In the RTL abstraction, these two are intertwined, and there is no semantic difference between, for example, a register used to meet timing closure and one being used to store computational state between clock cycles. This forces developers to always consider the effect on both timing and computation for any change they make, limiting the possibility to perform design space exploration. Some tools, for example Halide [16] build abstractions that separate the algorithmic description from the hardware mapping, allowing developers to iterate on one without affecting the other.

Another important thing to note about abstraction is that very few tools are limited to exactly one level. Verilog and VHDL, as mentioned, support mixing netlist and RTL-level descriptions. HLS can be used to synthesize both an unmodified software program, and a program written specifically for HLS with a plethora of pragmas to tweak the output. In many ways, the most interesting thing about abstraction is what levels one *cannot* reason at. Verilog and VHDL are always going to be bound by the "tyranny of the clock" [17], and HLS will never give full control over at which clock cycle each operation is executed.

A central idea underpinning the design of Spade is therefore that for productive and high-performance design, designers must be able to reason at many levels of abstractions in the same project. The abstractions are primarily designed to force the user to *explicitly* deal with problematic aspects, rather than hide details. As a third design consideration, the included abstractions are designed to be "low-overhead", meaning that they have little or minimal overhead over a lower level of abstraction. This principle is a key design principle of both C++ [18] and Rust [19] which is a big part of the reason that those languages manage to give both high performance and high productivity.

Ultimately, it is unlikely that there will ever be a one-size-fits-all language, and there will almost certainly never be a *perfect* abstraction. Which abstraction, and by extension, tool, to use depends on the goal of the project and the expertise of

the developers involved. In accelerator design, it is common for hardware designers to work with domain experts to build hardware, and in such cases finding a common ground that suits both, or finding a way to interoperate different levels of abstraction is a key to success. Finding a way to interoperate tools at different levels of abstraction is an important contribution of Paper G.

2.3 Lowering

Once a design is written in a suitable language, it must be converted to hardware. As most back-end tools today only accept Verilog or VHDL as the input, any higher-level languages must first be transformed into Verilog or VHDL. For new HDLs, which will be referred to as neo-HDLs throughout the rest of the dissertation, the process depends on the abstraction level. For languages based around the RTL abstraction, this is primarily a source-to-source transformation, while languages with higher levels of abstraction have more complicated compilers. HLS for example often starts by compiling the source language using its standard compiler, for example, Clang and LLVM for C++. Then, a sequence of lowering steps are performed to transform the compiled artifacts to hardware. More details on the inner workings of HLS tools are given in Chapter 4.

As there are a plethora of new languages and tools available for hardware design, there are several projects which attempt to build common infrastructure for hardware compilation. An important such project is CIRCT [20] which defines a large number of hardware-specific intermediate representations and lowering steps between them which is used by an increasing number of hardware compilers, both for neo-HDLs and HLS.

2.3.1 Synthesis

With an RTL description available, the next step in the lowering process is to transform it into basic logic gates that the target platform supports, a process called synthesis. In the case of an FPGA, the output of synthesis is a set of look-up tables, registers, memories, and special purpose blocks such as multipliers. In the case of an Application-Specific Integrated Circuit (ASIC), the primitives are standard cells generally including NAND gates, inverters, registers, etc. Synthesis also performs several optimization steps in order to reduce resource usage and improve the maximum frequency that the design can run at. Typically, the synthesis process goes through several internal steps, starting with transformations from Verilog into generic and technology-independent logic cells. These logic cells are then transformed into technology-specific cells such as the specific NAND-gates that can be manufactured in the target process, or the logic elements that make up an FPGA. Throughout this process, optimizations are also performed which reduces the number of logic cells that are required. Most of these optimizations perform boolean optimization to reduce the number of logic gates required, but sequential optimization like retiming and automatic clock gating can also be performed. The output of the synthesis process is a netlist, essentially a list of primitives to use, and how they should be interconnected.

2.3.2 Place and Route

Notably this netlist produced by synthesis does not mandate the physical location of each gate and connection, instead this is decided by the step in the compilation process: place and route. This is the first time in the design process where the real world truly comes into view because now each logic cell needs to be placed in a *physical* location on the chip and overlap between cells and/or wires is physically impossible and therefore strictly prohibited. Place and route are not trivial tasks, especially for large designs consisting of millions, or even billions of gates which have to be physically placed and connected such that they do not interfere with each other while also ensuring that signals arrive fast enough for the hardware to behave as expected. In order to make the problems tractable, placement is typically done first, using only heuristics for connections between cells. Once the cells have been placed, the connections between them are decided, again ensuring that the resulting layout is compatible with the real world. All three problems of synthesis, placement, and routing are NP-hard, so heuristics and stochastic algorithms are used to make the problems tractable.

When targeting FPGAs which have fixed locations of logic cells and interconnect, place-and-route is somewhat simpler than when targeting ASICs where the process consists of several sub-steps including floor planning, placement, clock tree synthesis, as well as global and detailed routing. The details of those steps are out of scope for this dissertation, however. [4]

2.3.3 Post Place and Route Analysis

After place and route, some static analysis is performed to ensure that the circuit does not violate any electrical rules and behaves as expected. This includes design rule checks, antenna checks, and static timing analysis. Static timing analysis is of some importance to this dissertation as it tells the designer what the maximum frequency the design can run at before starting to produce incorrect results. After static timing analysis, designers often have to make changes to improve timing and meet the performance requirements for the project which can be a tedious, and sometimes complex process.

In FPGAs and some ASICs, the lowering steps are usually fully automated by tools with the user only making changes in the input language. In high budget ASICs designs however, back-end design engineers often manually make changes to intermediate results.

2.4 Validation and Verification

In parallel with the design process discussed so far, a significant amount of engineering effort is spent on verification and validation: ensuring that the hardware behaves as expected. Depending on the cost of bugs slipping through to production, the thoroughness of verification varies. In a small FPGA project where it is

[4] Curious readers can find more information in the OpenLane documentation https://openlane.readthedocs.io/en/2023.09.07/flow_overview.html

relatively easy to iterate on the design and change the bitstream, simply simulating the circuit for common inputs is often sufficient. On the other hand, when building a large ASIC where retroactive changes are impossible and any design bugs that slip through require re-manufacturing the chip, a great deal of effort is spent on verification.

Simulation and testing can be used to check that a design meets its specification, but exhaustive testing of all possible inputs is generally infeasible for nontrivial designs. Formal verification on the other hand uses mathematical reasoning to *prove* that the implementation matches the (formal) specification [21]. The fact that formal verification only proves adherence to a specification is an important caveat, as errors in the top level specification will not be caught [22]. As an interesting aside, the Verilog simulation semantics as written in the specification are inconsistent, which makes a formal model of the language *impossible* to define [23]. While this is clearly a problem that has been worked around since hardware designs have been formally verified, it does hint at a problem that can be addressed with new HDLs, though this is not addressed in this dissertation.

While simulation or verification are not a focus of this dissertation, the debugging process that follows is. Once an issue is identified through simulation or verification, a designer must debug the design to identify the root cause of the issue and address it. This is typically done by looking at traces (waveforms) that show the sequence of signals that led to the incorrect behavior. The primary interface for viewing these traces is a waveform viewer which is discussed further in Chapter 5.

Verification can be also be extra-functional. As opposed to functional verification which is used to ensure that the circuit is error free and meets the specification [24], extra-functional verification identifies bigger picture system qualities including performance, throughput, bus-utilization, power consumption, and manufacturability. Waveforms provide a great deal of detail about the exact behavior of the chip, but getting an overview of bigger picture behavior such as bus utilization is often difficult. For this, other tools such as the Waveform Analysis Language (WAL) [12] are often a better fit. This too is discussed in more detail in Chapter 5.

Chapter 3
Hardware Description Languages

As mentioned in the introduction, Hennessy and Patterson predict a golden age of hardware design brought on by the need for more specialized accelerators to meet compute demands. However, for this golden age to start, there is a need for better, more software-inspired tooling [1]. A key to achieving this vision is new and improved hardware description languages as they have the potential to improve correctness, enable more code reuse, and improve performance [13].

By far, the most commonly used HDLs are Verilog and VHDL with a combined market share of around 90% for FPGA designs and 85% for ASIC designs in 2022 [9]. However, as discussed in the introduction, those languages are rooted in language design from the 1980s, operate at a very low level of abstraction, and are developed very slowly. This means that there is a wide variety of proposed neo-HDLs. Some languages like Chisel, [25], SpinalHDL [26], Clash [27], Amaranth [28], and BlueSpec [29] have seen some industry adoption. Beyond those, there are many languages that have yet to see broad adoption but bring several new ideas to the table. In total, the author is aware of more than 50 novel HDL projects [5].

3.1 A Taxonomy

With such a diverse field, it is worth spending some time analyzing the ecosystem and finding a taxonomy that can categorize and differentiate the existing projects. The current HDL ecosystem can be relatively well categorized by three parameters: whether a language is embedded or standalone, which abstraction level it operates at, and the expressive power of its type system. The following sections describe each of these categories and are followed by a survey on the HDLs that are most relevant to this dissertation.

3.1.1 Embedded and Standalone HDLs

Many modern HDLs are not standalone languages, but rather use an existing software language as a base. The most common way to do this is for the hardware description to be done through instantiation of dedicated classes [6] which represent hardware primitives, user-defined modules, and their interconnection. In this way,

[5] https://gitlab.com/TheZoq2/list_of_hdls/

[6] Or something equivalent in the host language

the host language can be used to provide very powerful meta-programming capabilities in the HDL, and the HDL can take advantage of a lot of the ecosystem of the host language. Typically, host language dependency management is used when present, and existing IDEs can be used for hardware description without any effort put in by HDL developers or users. There are many examples of this sort of embedded language: Chisel [25] and SpinalHDL [26], which are embedded in Scala, as well as Amaranth [28] which is embedded in Python, are perhaps the most well-known modern HDLs. In addition to those, there are many more languages embedded in Rust, Ruby, Haskell, and more. As constructs in the host language are reserved for meta-programming, the embedded languages can sometimes feel like "second-class citizens". For example, a Domain-Specific Language (DSL) "if-statement" in Chisel is written as `when (cond) { ... }.otherwise {...}` which hurts language discoverability and means that tools like auto-formatters reformats them in unexpected ways. In addition, error messages in the DSL are runtime errors in the host language which often hurts the clarity of the error messages.

This class of languages has several names, Chisel [25] calls itself a "Hardware Construction Language (HCL)", while Amaranth [28], calls itself a "Hardware Definition Language". To avoid ambiguity, these languages will be referred to as HCL throughout this dissertation. Further, the language such an HDL is embedded in will be referred to as the *host language*, and the HDLs as the *embedded language*.

Another less common method of working on top of an existing software language is to compile a subset of the language to hardware. Perhaps the most well-known language in this category is Clash [27], which compiles a large subset of Haskell code to hardware, with some supporting libraries for representing hardware constructs. Another example is PipelineC [30] which compiles a subset of C to hardware, though unlike Clash, not all PipelineC code behaves the same in hardware as the input C code, instead, PipelineC is a separate language that, where possible, tries to have the same behavior as C.

Finally, there are of course HDLs which are largely independent of any existing software language, though these often draw heavy inspiration from existing languages. Spade falls into this category, being a standalone language with Rust as a primary source of inspiration.

3.1.2 Abstraction Level

As mentioned in Chapter 2, abstraction is one of the key things a language designer has at their disposal for language design. Abstraction can hide unimportant details, force developers to be explicit about important details, and reframe problems in a way that makes them easier to reason about.

No contemporary HDLs that the author is aware of operate purely at a level below RTL, but many do not lift the abstraction level above that. This is the category that VHDL and Verilog fall into where it is joined by a few more neo-HDLs. One example of this is Veryl [31] which explicitly aims for interoperability with and gradual integration in existing SystemVerilog code bases by not adding too much abstraction on top. These languages also often have an explicit goal of their resulting generated Verilog being very close to the input.

The next level of abstraction is perhaps the largest and most diverse. Languages in this category have similar base semantics to Verilog and VHDL using RTL as the primary underlying abstraction. However, unlike the previous category, these languages provide additional abstraction to shift the reasoning from pure RTL. An example of this is languages with built-in support for describing pipelines in a structured way such as Spade, TL-Verilog [32], and ROHD [33]. Similarly, Filament [14] and SUS [15] provide facilities for reasoning about the timing of signals. In addition, many of the established embedded languages also fall in this category of "RTL+" languages. Their host languages provide the ability to use powerful meta-programming to generate hardware and to build additional abstractions, but at the end of the day, the resulting hardware description is in RTL.

Beyond "RTL+" there are languages which completely move away from the RTL level and often *require* reasoning at a higher level of abstraction. This category is broad and could even include HLS. For this reason, it is hard to form neat categories for them beyond having a non-RTL level of description. Some examples of these languages include

- DFiant HDL [34] which uses dataflow as the primary abstraction for hardware description, but also supports both RTL and netlist descriptions [17].
- Silice [35] which includes an imperative language for describing algorithms combined with features required to accurately describe hardware
- BlueSpec [29] which describes hardware using rules and actions
- PipelineC [30] which does auto-pipelining of most circuits and supports some sequential hardware description.
- CoHDL [36] which uses co-routines to describe FSMs in an imperative way.
- YieldFSM [37] which extends Clash with a generator-based description of FSMs.

3.1.3 Type Systems

Another way to differentiate languages is by their type system. Some languages including Verilog have very few types: everything is bit vectors which may or may not be signed. Any further interpretations of the values is unspecified in the language itself, somewhat akin to dynamically typed languages in software.

Many languages have stronger type systems than that, allowing definitions of compound data types as well as limiting which operations can be performed on which values based on their type. A lot of HDLs support this, including VHDL, SystemVerilog, and many of the HCLs. It is worth pointing out that HCLs really have two type systems: the type system of the host language as well as the type system of the embedded language. Scala for example is very strongly typed, but the hardware generated in Chisel [25] and SpinalHDL [26] which are embedded in it typically operates solely on bit vectors and compound types.

In the functional programming community, which generally has powerful type systems, compound types are known as product types and are often joined by *sum types* or *tagged unions*. Type systems that support both are called algebraic [38]. Sum types allow types which have multiple interpretations of the underlying bits, with a tag that specifies which is the active interpretation. In hardware, a good use case for these is valid signals, which are bundled with the data they validate.

The tag specifies whether the payload can be interpreted as the underlying data, or is undefined and should not be read. Languages with tagged union support allow expressing this relationship explicitly, and forces the user, at compile time, to handle the fact that the underlying data can be invalid. Since they are a central part of functional programming, Clash [27], which supports a very large subset of Haskell, also supports sum types. Similarly, Spade [39] and RHDL [40] being inspired by or based on Rust also supports sum types. The use of sum types for representing valid bits and the advantages it provides is the focus of Paper B.

So far, all the type systems discussed are focused on adding more semantic meaning to individual bit vectors, but some languages have type systems that support reasoning about other properties.

One such example is substructural types, which allow reasoning about the lifetime and state changes of objects [41]. In the software world, this is often used to model files being opened or closed and locks being unlocked or held, etc. In Spade, this is used to model ports in a way that ensures that the drivers of all signals are unique. In the Dahlia [42] language, affine types are used to model memory resources which ensures that memories are partitioned and accessed efficiently.

When designing hardware, the timing of values in relation to each other is also a very important property, and as such there are several languages that allow reasoning about timing in the type system. The prime example of this is timeline types which were introduced by Filament [14]. Timeline types allow the type system to ensure that each instance of a module is only used once in every clock cycle and that the inputs are valid and in sync when the module is used.

Finally, some type systems allow so-called higher-order functions that accept, transform, and produce other functions. This is typically used heavily in the functional programming paradigm and some HDLs which stem from this paradigm inherit that support. For example, Clash's [27] standard library includes functions for defining finite state machines where one of the function parameters is the transition function from the current input and state to the next state and output. ShakeFlow [43] uses higher-order functions to customize the behavior of combinators by allowing the user to specify a function to transform or filter the contents of the stream.

3.2 Existing HDLs

With a taxonomy established, we can start to categorize and discuss existing languages. Figure 4 lists details on most [7] of the HDLs known to the author and where they fit in the proposed taxonomy. Abstraction is split into *RTL* for languages that have a similar execution model to Verilog, *RTL+* for languages that are still an RTL description at the core, but provide additional low overhead abstractions on top, and *other* for less common abstractions. Since all embedded languages allow building new abstractions through metaprogramming in the host language, these are categorized by the primary abstraction used in the DSL. The type system of each language is categorized by its expressive power, grouped into Verilog-like

[7] Languages which are unmaintained or undocumented have been filtered out.

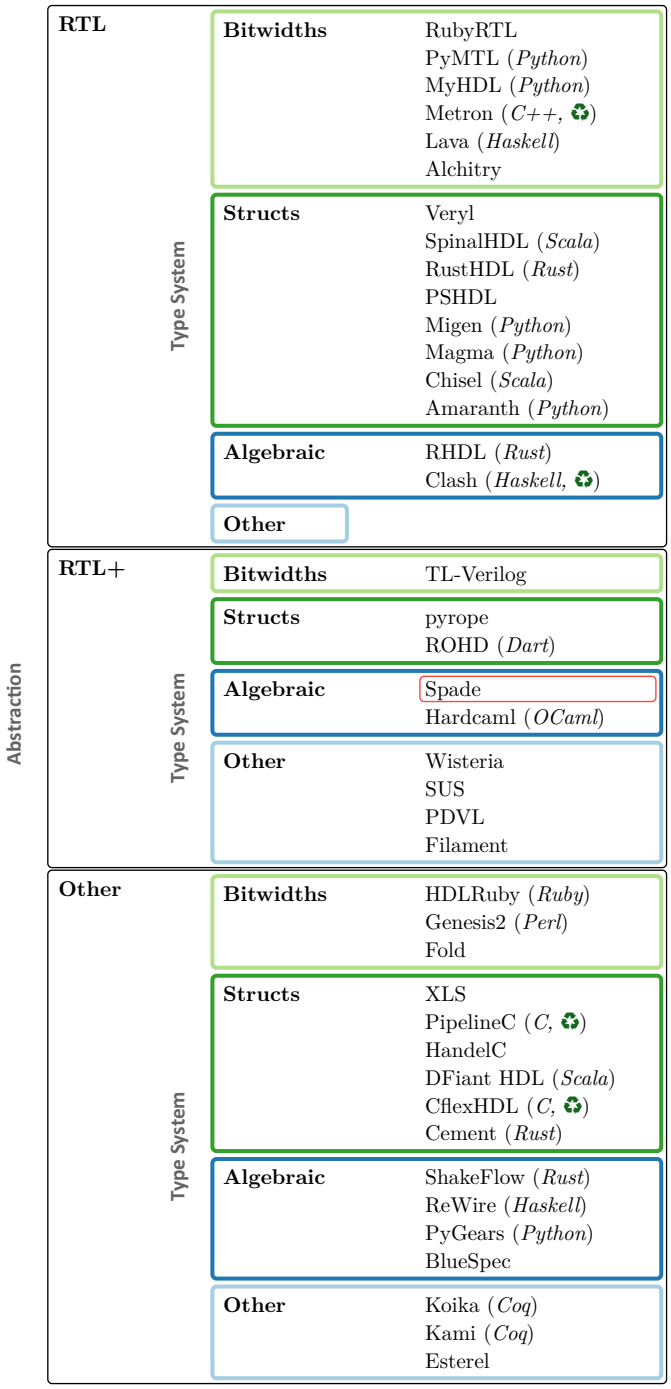


Figure 4: List of HDLs categorized by their position in the proposed taxonomy. The parenthesized languages are the host language for embedded HDLs. Languages which borrow semantics instead of embedding are marked by ♻

Table 1: Number of HDLs at each point of the proposed taxonomy. Spade being a standalone RTL+ language with full algebraic data types is outlined red.

		Standalone			Embedded		
	Abstraction	**RTL**	**RTL+**	**Other**	**RTL**	**RTL+**	**Other**
Types	**Bitwidths**	3	1	1	3	0	2
	Structs	2	1	4	6	1	2
	Algebraic	1	1	1	1	1	3
	Other	0	4	1	0	0	2

type systems which encode little more information than bit widths (*Bitwidths*), type systems that support compound types such as records or structs (*Structs*), *Algebraic* for languages with support for sum types, and *other* for languages which do not fit into either category, in general because their type systems are more powerful. Table 1 provides a summary of the number of HDLs in each category.

The field is clearly vast with languages at nearly every possible point in the taxonomy. As the list of HDLs is long, there is neither time nor space to go into detail on all of them. Instead, the rest of this section highlights some of the commonly used languages and some languages which provide especially interesting features.

3.2.1 Chisel, SpinalHDL, and Amaranth

Chisel, SpinalHDL, and Amaranth are among the most widely adopted neo-HDLs. All three fall in the category of embedded languages with powerful metaprogramming features but with few non-metaprogramming abstractions and type systems that are largely restricted to the host language. Chisel and Spinal are both embedded in Scala which gives them a powerful static type system at design time, while Amaranth is embedded in Python which is dynamically typed [8].

The hardware description in all three languages is primarily at an RTL level of abstraction, though with the powerful metaprogramming capabilities that they provide, it is possible to build much higher-level abstractions in the language. For example, VexRiscv, written in SpinalHDL, provides a highly configurable RISC-V core that can be constructed by simply passing a list of "plugins" which includes instruction set extensions, branch predictors, caches, etc. During *evaluation*, the underlying Spinal code then converts this down to an RTL-level description. Similar systems exist in both Chisel and Migen (a precursor to Amaranth) for generating SoCs.

These languages also come out of the box with abstractions that go beyond the Verilog level of abstraction. Some examples of this include all three supporting clock domains and clock domain crossing, Amaranth supporting native data streams for ready/valid handling, and Spinal having native support for pipelining.

[8] With optional type annotations

3.2.2 Clash

Clash is a somewhat unique language in the sense that it compiles a large subset of Haskell directly to hardware. While compiling a software language to hardware in itself is not unique, it is usually done as HLS which hides many of the implementation details. Clash instead adds a few hardware-specific types to Haskell for reasoning about signals, clock domains, and state machines which the dedicated compiler is able to compile to hardware. Signals themselves hold normal Haskell types which means that the full Haskell language can be used to describe computations to perform on the signals.

3.2.3 Filament

Filament [14] uses *timeline types* to reason about structural hazards at compile time. Modules are parameterized by events which are used to define the availability intervals of inputs and outputs. When invoking a module, the compiler statically ensures that the inputs are available at the required intervals which allows correct-by-construction composition of modules with complex timing requirements. Filament also includes generative programs which can be used to write code that is parameterized over its timing behavior [44]. For example, a single divider implementation can be instantiated as a pipelined divider, a combinational divider, or a resource-sharing multi-cycle divider.

3.2.4 DFHDL

DFiant HDL [17, 34] is an embedded HDL in Scala that uses dataflow as its primary abstraction. In dataflow circuits, a design is specified in terms of streams of tokens, not in terms of specific wires and registers. This allows operations to execute whenever their inputs are ready instead of at precise clock cycles that must be scheduled by a designer. In addition, the dataflow abstraction differentiates registers by their use, being explicit about which registers are used for constraints to meet timing, which hold design state, and which are used for cycle-accurate operations like timing.

In addition to the primary dataflow-based abstraction, DFHDL also allows writing designs at the RTL-level and event-driven level, which allows mixing abstractions as appropriate for the project. Unlike most HCLs, DFHDL hooks into the compiler to reuse the normal Scala primitives for language constructs such as if-statements, avoiding many of the usability issues that come with embedded DSL.

3.2.5 BlueSpec

BlueSpec [29] moves away from the RTL abstraction that most other HDLs use. Instead, sequential modules in BlueSpec consist of internal state which are either submodules or plain registers, “methods” which define the module’s input and output logic, and “rules” which define the module’s internal behavior. Both methods and rules consist of a guard that specifies under what conditions the method or rule can be executed and a list of statements which are executed to completion if the method or rule is triggered. Guards can also be implicit if the

statements to be executed contain method calls which in turn have additional rules. The exact scheduling of rules and methods is determined by the compiler [45].

The BlueSpec HDL comes in two interchangeable flavors: BlueSpec Classic with Haskell-inspired syntax and BlueSpec SystemVerilog which as the name implies has SystemVerilog syntax. Both languages have the same semantics, and the type system in both is very similar to Haskell.

The actions-and-rules abstraction has been adopted by several other HDLs including Kami [46], Kôika [47], Cement [48], and cmt2 [49]. The latter extends the system with inter-rule temporal relationships and multi-cycle rules, which allow the description of latency-sensitive composition.

3.2.6 Verification Focused Languages

Many existing HDLs are primarily focused on improving the *design* phase of hardware development. However, a few languages put greater emphasis on verification with compilers and abstractions built to ease this process. Some examples of this include ReWire [50], PDVL [51], Silver Oak [52], and Kôika [47]. As verification is not discussed further in this dissertation, the details of these languages are not discussed here.

3.3 Contributions

Spade fits in the proposed taxonomy as a standalone strongly typed HDL. This naturally sets it apart from all the embedded HDLs. At the core, it uses an RTL abstraction, though it includes several high-level abstractions on top. Its type system is more powerful than most HDLs, except for Clash and BlueSpec. However, Clash is somewhat limited by its Haskell compatibility and BlueSpec uses actions and rules as the fundamental abstraction rather than RTL.

3.3.1 Pipelining

One of the primary contributions of Spade is language level support for pipelining which separates the computations being performed in a pipeline from the placement of pipeline registers. By making pipelines a native language feature, developers and the compiler alike can reason about the latency of pipelines to guarantee that they are used correctly. This in turn allows “fearless retiming”, where users can change the structure of one pipeline in the design and be forced by the compiler to adjust the rest of the design to accommodate that change. Pipelining is not a wholly new concept, TL-Verilog [32] includes a very similar language construct that splits computations into pipeline stages. However, the latency of pipelines is not exposed externally, which means that the compiler cannot guarantee correctness of pipelines instantiated in other pipelines, losing out on the fearless retiming. Pipelining can in some ways be seen as a weak form of Filament’s timeline types [14] as it can be used to reason about input-output latency, but unlike Filament there is no way to reason about initiation intervals. On the other hand, the Filament language is primarily an academic project that focuses specifically on timeline types which makes it less useful as a general HDL.

SpinalHDL gained support for pipelining after Spade. Unlike Spade, where pipelining is part of the syntax, the Spinal implementation is more verbose, requiring the developer to explicitly define stages and binding of variables to those stages. However, it provides the same guarantees as the Spade system does, and in addition, includes support for ready/valid handshaking between pipeline stages.

ROHD [33] has also received support for pipelines which is similar in style to Spade with the usual caveats that come with an embedded HDL. Similarly to TL-Verilog, ROHD does not directly expose the latency of pipelines in the head of modules, which means that sub-pipelines have to be handled correctly by the user. The metaprogramming facilities do allow building the same abstraction that Spade provides, though it would be done in an ad-hoc manner and different projects may end up with different incompatible implementations.

SUS [15] achieves similar guarantees to Spade pipelines with its latency counting system. Latency counting keeps track of the latency of individual variables, rather than grouping them into stages.

Finally, PipelineC [30] supports auto-pipelining which also guarantees correctness when refactoring, though because it uses auto-pipelining, the exact timing is not something the user can control.

3.3.2 Type System

The Spade type system is also a unique feature in the HDL landscape. It is roughly as powerful as the Rust type system, excluding borrow checking which is a software-specific abstraction. Unlike most HDLs, it includes support for sum types, or tagged unions with the language-level constructs to support them. This allows, among other things, type-safe valid signals on data and easily expressible custom instruction formats. The type system also includes support for *ports* which are collections of bidirectional wires that can be used to model well-defined interconnection between ports. Using linear types, the type system guarantees that these ports are used correctly, i.e., that every signal is driven exactly once.

Both BlueSpec and Clash include a powerful Haskell-derived type system that in many ways is more powerful than the Spade type system. However, as far as the author is aware, neither language includes support for ports.

3.3.3 Tooling

While a language itself can do much to improve developer productivity, so can the tooling that surrounds it. Embedded languages benefit from the tooling in the host language which provides editor integration, compilers, and package management facilities. In a standalone HDL however, these tools must be built for the language. Spade includes a package manager and build tool called Swim, a language server client, a work-in-progress documentation renderer, and a work-in-progress auto-formatter. Some experiments have been done to extend the Spade build system to support more languages [53].

In addition, debugging is an essential step in the design process that can be aided by tools. Paper C describes an integration between Spade and the Waveform Analysis Language (WAL) [54] which allows users of libraries to use analysis scripts

bundled with libraries with minimal effort. In addition, the Surfer waveform viewer described in Paper D was originally built specifically for Spade to enable easier debugging of signals with complex hierarchical types.

3.3.4 Effects on the HDL Ecosystem

Spade has been in development for over four years and meanwhile neo-HDL ecosystem has undergone rapid changes. In fact, many of the HDLs in Figure 4 were created after Spade. This means that Spade has been a direct inspiration for some languages, most prominently, the latency counting system in SUS is in part developed to address some of the shortcomings of the pipelining system in Spade [15]. Other languages gained pipelining features after Spade, including ROHD and SpinalHDL though it is not clear if Spade served as inspiration for these, or the idea was simply developed independently.

Some ideas were also developed in parallel. For example, an early preview of the timeline types in Filament, which addresses timing in a much more rigorous way than Spade's pipelines, was presented at a workshop at FCCM in 2022. This workshop ran in parallel with the first workshop where Spade was presented.

Chapter 4
High-Level Synthesis

Hardware Description Languages provide a lot of control over the hardware that gets generated, but since the description is at such a low level, it takes significant effort and knowledge to build hardware using HDLs. In an attempt to solve these problems, a significant amount of resources have been invested in developing so-called High-Level Synthesis tools which automatically translate high-level software programs into hardware, without the programmer having to concern themselves with the low level details of exactly how that hardware is constructed. There are a large number of research and commercial HLS tools, many of which are listed in [55]. Perhaps the most well-used tool is AMD Vitis HLS [9] which accepts C, SystemC, and, C++, code and automatically translates it into Verilog or VHDL for synthesis on AMD FPGAs. Other commercial HLS tools include Catapult C which also translates C, C++ and SystemC into hardware, and Matlab HDL coder which as the name implies translates Matlab code into hardware. There are also some academic and open-source alternatives. Bambu [56] is a framework for research on HLS which like most commercial counterparts accepts C, C++, and SystemC as input. It uses the GCC compiler as a frontend which enables the use of existing optimization passes that software compilers already support. SODA [57] is an open source high level synthesis framework focused on synthesis from domain-specific languages for accelerators. It uses MLIR [58] to generate LLVM IR that is fed to Bambu for synthesis. LegUp [59] was another open-source offering also supporting synthesis of C and C++. It generates hybrid architectures containing both a MIPS soft processor and custom FPGA hardware. Which functions to run on which hardware is determined automatically based on the suitability for hardware acceleration. Since its publication, LegUp has been commercialized and subsequently acquired by Microchip. It is now known as SmartHLS [60] and as far as the author is aware, source code is no longer provided.

The tools discussed so far all use static scheduling which means that operations are scheduled at compile time based on worst-case assumptions. This leaves performance on the table if there are runtime-dependent dependencies [61]. Dynamatic [62] is an open-source HLS tool that uses dataflow to perform dynamic scheduling in order to improve performance in data dependency-heavy applications. Like most HLS it uses C and C++ as input, and as of version 2.0, it uses the MLIR infrastructure for its implementation.

[9] Formerly Vivado HLS

Most HLS consumes standard software languages with C and C++ being the most common option. Another commonly used option is Matlab HDL Coder, which as the name implies, compiles Matlab functions as well as Simulink models to hardware [63]. There are also a few languages which are written specifically for HLS. One such alternative is Dahlia [42] which uses time-sensitive affine types to model memory partitioning and access patterns in order to make the performance of the resulting HLS code more predictable. Dahlia code gets compiled to C++ code which is fed into Vitis HLS. XLS [64] is a high level-synthesis toolchain which, while it can be used with C++, also comes with a purpose built domain specific language: DSLX [65]. It is designed to explicitly mimic Rust where possible, with some changes for better mapping to hardware. These changes include allowing arbitrary bitwidths, requiring objects to be of fixed size, and requiring a statically known call graph.

4.1 Typical Static HLS Flow

Modern HLS tools like most modern compilers are split into several phases which gradually lower the source language down to hardware via a series of intermediate representations. The HLS frontend is responsible for the initial transformation of the input language and is almost always done by an existing software compiler when one exists. For C and C++, Clang which compiles to the LLVM intermediate representation is a popular choice used by Vitis [66] and LegUp [59]. Bambu which also supports LLVM is frontend agnostic and can also use GCC as the frontend [67]. More recent HLS such as SODA [57] are gravitating towards using MLIR [58] as the frontend which gives more flexibility in the input language and more re-use of hardware-specific optimizations, especially via the CIRCT [20] dialects. HLS tools also often apply some of the optimization passes included in the software compiler since many of them are advantageous for hardware and software alike. The exact effect of software optimizations on the resulting hardware varies, and significant performance gains can be made by fine-tuning which optimization to run in which order [68].

Until this point, the compilation has largely been the same as for software, and for the compilers that use LLVM or GCC, the internal representation is still the corresponding IR. This IR is fed into the back-end which performs target and hardware-specific optimizations. AMD does not publish exactly what happens at this stage apart from “FPGA specific optimization/lowering” followed by the “back-end” which does “scheduling and beyond” [66]. Bambu is more public with what steps are performed [67]. The middle-end performs some software-like optimizations as well as some hardware-specific optimizations such as transforming multiplications into shifts and adds, as well as bound and range analysis for reducing the number of bits. In the back-end phase, the tool generates a hardware architecture from the results in the previous steps. This involves multiple steps including allocation which selects which resources to implement, scheduling which selects when each step is to be executed, and finally binding which maps the scheduled operations to hardware resources. The end result is a statically scheduled

circuit consisting of control logic implemented as a state machine and a datapath that is a MUX-based architecture with optimized data types.

4.2 Dynamic HLS

The schedule generated by most HLS tools is static – each operation in the program is executed by a specific compute unit at a known time. This means that it cannot exploit parallelism in code with data-dependent memory or control dependencies, a problem that is addressed by Dynamatic [69]. The frontend of Dynamatic is similar to that of static HLS, it uses MLIR with the Polygeist C++ frontend [70]. The backend differs more, however. Instead of generating a static schedule at compile time, it generates a dataflow circuit that does scheduling locally in each execution unit, with handshake signals that communicates readiness and validness between units. Execution units in the generated circuits can be both compute units such as arithmetic, and control units such as forks, joins, merges, and FIFOs. The resulting circuits are on average larger than the corresponding statically scheduled circuits but can run significantly faster if there are unpredictable memory dependencies or irregular control that cannot be predicted at compile time flow [69].

4.3 Accelerator Design Languages

There is a big leap in abstraction between even high-level HDLs and HLS which has naturally led to some projects that attempt to fill the gap. Halide [71] is a domain-specific language for image processing that decouples the algorithmic description from its scheduling, which allows high-performance processing while retaining code clarity. While initially intended for parallel processors and GPUs, the language has been extended [16] to also support compilation to hardware with similar advantages.

Allo [72] follows a similar approach of decoupling algorithmic description from hardware customization, and unlike Halide, it is not restricted to image processing applications.

Spatial [73] is a domain-specific language for accelerator design embedded in Scala. The algorithmic description in Spatial is similar to HLS in the sense that there is no description of precise timing, and that it encourages accelerator description in terms of nested for-loops. Unlike HLS however, Spatial gives users explicit control over memory hierarchies, performs pipelining, memory banking, and memory duplication without the need for explicit pragmas. In addition, it supports automated design space exploration using active machine learning.

4.4 Contributions

HLS is generally used as an end-to-end tool that synthesizes a full architecture from an input program. However for that to yield good performance, the tools typically need some additional input in the form of pragmas that control hardware specific details which requires hardware expertise, and in many applications the results

will still be far from a handwritten implementation by an experienced engineer. This means that the promise of domain experts being able to generate hardware is not fully realized, and at the same time, hardware engineers are forced to use a sub-par tool for their job.

We propose a different approach to accelerator design for dynamic programming-based optimal control where domain experts can use HLS to generate *efficient* hardware for a simulation model. This is possible thanks to the simulation models not being stateful, which means that the HLS tool can simply generate a feed-forward pipeline rather than an architecture that requires complex control logic. This fully pipelined simulation model can then be combined with a handwritten executor of the model which a hardware engineer can develop in parallel. The advantage here is that the exact behavior of this executor is much less dependent on details of the model, thus requiring much less iteration from the domain expert. At the same time, the control logic is much more complex, which means that hardware expertise is required for an efficient implementation. In Paper G we show how this can be employed to implement an accelerator that improves the fuel efficiency of hybrid electric vehicles.

When synthesizing pipelined simulation models, there are different considerations that must be made than in general-purpose HLS. No control logic needs to be generated, and because there are no feedback loops in the simulation model the allowed range of every signal in the resulting design is known assuming the range of the inputs is known. In addition, many optimal control applications can sacrifice bit-level accuracy with the software model, which leads to a reduction in resource usage through the use of fixed-point numbers. Therefore, we built Cinnabar which is introduced in Paper E and is built specifically for these constraints. It generates pipelined hardware from C++, replaces all floating-point operations with fixed-point operations, and uses range analysis to reduce integer word length. It can also be used to tune the fractional word length of signals in order to fine-tune the output error/resource usage tradeoff, though unlike the integer word length which is selected statically on a per variable basis, this is only done through simulation with a set word length for each variable.

Finally, when generating pipelined hardware from software, the software control flow must be converted to multiplexers in hardware. Whether this is done when generating pipelined hardware, and if so how it is done in existing HLS tools is rarely documented. Paper F presents several algorithms for performing this conversion along with some benchmarks for which algorithms perform well. The overarching conclusion is that the best algorithms to use depends on the application.

Chapter 5
Analyzing Simulation Results

An important step in any development work is debugging, which unlike language design, is an area where hardware design is in many ways ahead of software. Both simulation and formal verification tools produce traces (waveforms) that show all signal values throughout the simulation, allowing detailed retroactive debugging.

The primary interface to the waveforms is usually the *waveform viewer*, an example of which is shown in Figure 5. The user adds the signals they are interested in viewing to the list of viewed signals, and the value of each signal is shown over time as an individual trace, allowing the user to inspect how values in the design changed over time. Commercial simulators often come with their own built-in waveform viewers, but in the open source space, most simulators dump their signals in a waveform file which is read retroactively by a separate waveform viewer. For many years, the de-facto standard waveform viewer in the open source space has been GTKWave [74].

While waveforms allow designers to go into great detail to see the value of every bit in the design, they are not as powerful for higher-level reasoning. For example, in an HDL with a powerful type system, the values of individual bits or even bit vectors mean relatively little without knowing what they mean in the underlying type. When debugging a processor, it helps to know what instructions are being executed in a human-readable form rather than the assembled machine code. Similarly, when working with transactions in a protocol such as AXI, visualizing the individual AXI signals only tells part of the picture, what the developer is

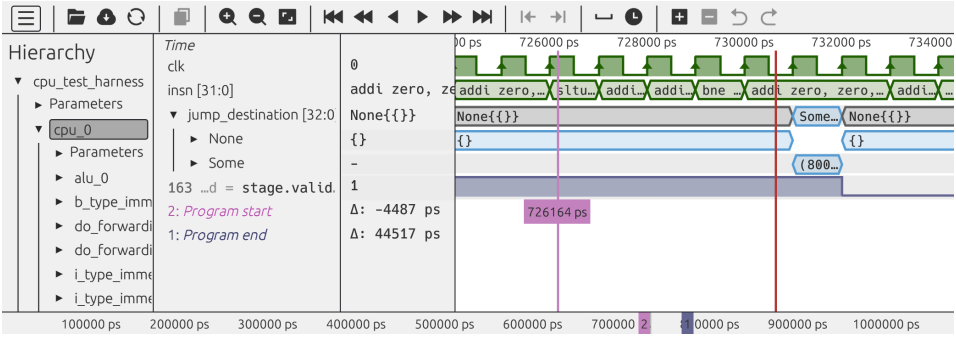


Figure 5: An example of the Surfer waveform viewer being used to debug a RISC-V core written in Spade.

```
(define wait 0)
(define packets 0)

(whenever (rising comp1.clk)
  (when (&& comp1.req (! comp1.ack)) (inc wait))
  (when (&& comp1.req comp1.ack) (inc packets)))

(print "Average delay: " (/ wait packets) " cycles")
```

Listing 1: A WAL program that computes the average delay between a request being issued and it being completed.

most interested in is most likely full transactions, unless they are debugging an issue with the AXI implementation itself.

Traditional waveform viewers usually make it difficult or impossible to highlight this additional information. GTKWave allows defining signal translators as look-up tables between individual signal values and a more human-readable representation, but defining these translators is cumbersome for small signals and impossible for complex signals when the bit width of the signal results in a lookup table that is infeasible to fit in memory. It also provides facilities for programmatic translation through an external program, but this too is limited to string values for individual timestamps which still makes debugging of hierarchical types and transactions that span multiple clock cycles difficult.

While waveform viewers are likely the most common way to interact with waveform data, it is not the only way. A prominent example is WAL [12], which is a programming language built around waveform analysis. A WAL program runs in the context of a loaded waveform and any variable access to variables not part of the program access signals in the waveform. The time step at which the signal value is fetched is determined by the *index*, a pointer to a time step that can be moved around programmatically.

A big advantage of programmatic waveform analysis is that one can easily extract meta-information that is hard to obtain simply by visual inspection. For example, Listing 1 contains a short WAL script that computes the average delay between a transaction being requested and it being completed. Finding this information by manually inspecting the `req` and `ack` signals is tedious, error-prone, and must be re-done every time a simulation is re-run. At the same time, such information can be invaluable when designing high-performance latency-insensitive circuits.

Waveforms are not only used for visualization but can also be used for documentation. Wavedrom [75] is one tool for this which allows specifying waveforms in a simple JSON format which are translated to an SVG that can be embedded in documentation.

5.1 Contributions

Surfer, presented in Paper D addresses many of the shortcomings of existing commercial and open source waveform viewers alike. It was originally built to aid in debugging Spade by providing a flexible translator system that can translate

from raw bit vectors back to their native Spade value. Surfer was built because GTKWave was not extensible enough to fit the needs of Spade, and to avoid this happening in the future, it was built to be *extensible* to support new use cases. This has resulted in several integrations with other HDLs, in particular, Chisel via TyWaves [76], Clash, and RHDL.

Also in the spirit of extensibility, Surfer exposes several ways for other applications to interact with and control the waveform viewer. This, together with the ability for it to run web browsers has led to it being used as a component in several commercial and open source projects.

As mentioned, there is some information that is much easier to capture programmatically than visually in a waveform viewer. For this, Spade was integrated with WAL, in part for similar reasons as Surfer — to make it easier to work with its high-level types. In addition, Spade provides package management and canonical representations for things like ready/valid interfaces, state machines and buses. By allowing the bundling of WAL analysis scripts with these libraries, users can get high-level performance analysis with minimal effort, making it easier to build high-performance hardware. This, along with the technical details on how the integration works is described in Paper C.

Chapter 6
Dynamic Programming for Optimal Control

The second track of this dissertation revolves around hardware that runs dynamic programming for optimal control, specifically focused on its use to optimize the fuel efficiency of HEVs. In this chapter, the necessary background for this topic is introduced, starting with optimal control.

6.1 Hybrid Electric Vehicles

HEVs combine a combustion engine with an electric motor in order to improve fuel efficiency and reduce emissions. The exact architecture of the drivetrain varies between different designs. In a series hybrid architecture, the electric motor provides all the torque to the transmission, with the combustion engine being used to drive a generator which provides power to the motor and battery. In a parallel architecture, the electric motor and the combustion engine are mechanically coupled before the transmission while a series-parallel or complex hybrids couple the two in the transmission and include a generator to charge or the battery with the combustion engine. The exact details of the architecture naturally change the model used for the vehicle, but the techniques discussed here can be applied regardless. For a more in-depth discussion, see [77].

One aspect that does have an effect on the topics discussed here is whether the vehicle is charge-sustaining or charge-depleting. In a charge-sustaining vehicle, the battery or capacitors are only used to store energy temporarily, either gained through regenerative braking or running the combustion engine [77]. This allows the vehicle to charge the battery when there is energy available such as when driving downhill, and spend that energy when the need arises, for example when accelerating or going uphill. The primary goal of the algorithms described here is to compute when to charge and discharge the battery while taking into account road conditions and traffic. In a plug-in hybrid, the battery can be charged externally from the electrical grid which further contributes to a reduction in emissions [78].

6.2 Optimal Control

Very broadly, the goal of optimal control is to find control signals to a dynamical system which takes the system from an initial state to a target state while optimizing some objective function [79]. It has far-reaching applications, naturally in physics but also in other fields such as economics [80].

Mathematically, a dynamical system is characterized by its dynamics

$$\dot{x}(t) = \boldsymbol{f}(x(t), u(t), t), \tag{1}$$

where x is the state of the system, t is time, and $u(t)$ is the control applied at time t. The goal of optimal control is to maximize or minimize the *cost-to-go* function

$$\boldsymbol{J}[u] = \int_0^T g(x(t), u(t))dt + h(x(T)), \tag{2}$$

where g is the running cost of applying $u(t)$ in state $x(t)$, and $h(h)$ is the cost of the final state [81].

As a more concrete example, which is central to this dissertation, optimal control can be used to improve the fuel efficiency of hybrid electric vehicles [82]. In this case, the state x is the state of the vehicle usually including at least its battery state of charge and its current velocity. The control inputs u usually then include the torque requested from the combustion engine and electric motor, the mechanical braking applied, and the current gear. The dynamics $\boldsymbol{f}$ then compute the next state of the vehicle from a model of the powertrain as well as from external factors such as weather conditions, other traffic, and the road layout including speed limits and hills. The running cost g is the cost of applying the selected inputs at each state, in the HEV case, this cost is a combination of fuel consumption and time to get to the destination. Finally, the terminal cost h is the cost of the final state which in a mild hybrid electric vehicle is set to a very high value if the final state of charge is lower than the initial state of charge.

Some optimal control problems can be solved analytically, but in this work, the focus is on numerical solutions. In addition, the focus is on discrete functions rather than continuous functions as discussed so far, which allows the use of dynamic programming [83] to find solutions [84]. When time is discretized, the dynamics of the system are described by

$$x_{k+1} = f(x_k, u_k, w_k), \tag{3}$$

where x_k is the state of the system at time k, u_k is the input applied at time k, and w_k models "disturbances" at time k, for example, road conditions in the vehicle modeling case. The goal is then to find a sequence of inputs $u_{0..N}$ which minimize the cost

$$J[u_{0..N}] = \sum_{k=0}^{N-1} g(x_k, u_k, w_k) + h(x_N). \tag{4}$$

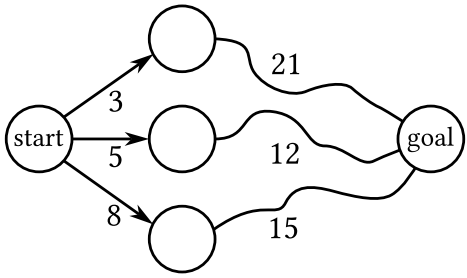


Figure 6: Graphical motivation for dynamic programming. If the cost of reaching the goal from the states reachable from *start* are known, then the cost of *start* is the sum of the cost of reaching a state plus the cost of reaching the goal from that state.

If the possible inputs are discretized such that they can only be picked from a finite set U, this problem can in principle be solved by evaluating every single input at every time step $k = 0, 1, ..., N-1$. However, in practice this quickly becomes infeasible as the number of model evaluations required is $|U|^N$.

6.3 Dynamic Programming

Dynamic Programming (DP) [83] is a general method for reducing the amount of computation required when solutions to sub-problems can be reused between computations. The general idea is to break problems down recursively into simpler sub-problems. For example, in Figure 6 finding the optimal path from the *start* node to the *goal* node can easily be done if the cost of the optimal path from the nodes reachable from the start node are known. Then the cost of each path out of the start node is the cost of the path to each node plus the cost of reaching the goal node. This strategy is then applied recursively, starting from the goal node whose cost is trivial, and working backward to the start node.

DP is a general technique that can be applied to a broad set of problems. The key to enabling its use in optimal control is the observation that costs are additive. This means that if we have already found the minimum cost of reaching the final state from state x_{k+1}, and we can reach that state from state x_k with some input u_k, then the cost of applying u_k in x_k is $g(x_k, u_k) + c_{x_k}$. This in turn means the cost of any state x_k need only be computed once and can be reused by any path going through the state. There is still one more problem however: If the possible states are continuous, then it is very unlikely that two paths will pass through the same state, meaning that while re-use is possible in theory, in practice it is unlikely to happen. For this reason, in order to apply dynamic programming to optimal control, the state space must also be discretized into a set of finite states denoted X_N for the possible states at time N. To accommodate this, the dynamics function f must also be adjusted to only return valid states.

Under these constraints, Algorithm 1 can be used to compute the cost to go at each state in the state space. The algorithm maintains a "cost to go" map which is gradually populated with the cost of reaching the final state from previous states. This map is initialized to contain the terminal cost of each state at the

Algorithm 1: Recursive implementation of the dynamic programming algorithm for optimal control.

```
fn dp():
    cost_to_go = ∅
    initialize cost_to_go with cost of all final states x ∈ X_N
    for t in N − 1 ... 0:
        for x in X_t
          for u in U
              x_{t+1} = f(x_t, u_t)
              c_local = g(x_t, u_t)
              c_total = cost_to_go[x_{t+1}]
              if c_total < c_best
                  c_best = c_total
          add x_t = c_best to cost_to_go
```

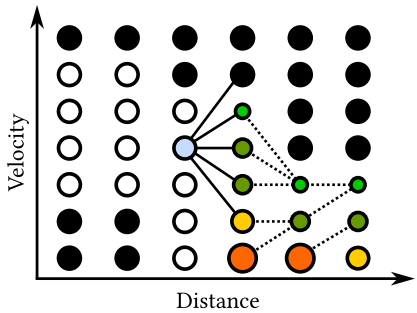


Figure 7: Visual representation of the principle behind Algorithm 1. The size of the states indicate their cost, dashed lines indicate the optimal input in the states in which it is known, and solid lines indicate inputs to evaluate in the current state. Black states indicate states disallowed by constraints such as speed limits.

final time step N. Then the computation is performed "backward", starting at time $N-1$, computing the cost of applying every input in every state. Figure 7 shows a graphical example of the working principle of the algorithm where the size and color of nodes represent their cost, dashed lines indicate the optimal input in the states for which it is known, and solid lines indicate inputs to evaluate in the current state. The black nodes are disallowed states, for example, those set by speed limits.

6.4 Hardware for Dynamic Programming

The number of model evaluations required to execute Algorithm 1 is

$$N \times |X_n| \times |U| \tag{5}$$

which is a significant reduction compared to the $|U|^N$ required without dynamic programming. However, as the number of time steps, states, and inputs grow, a

large number of evaluations is still required, therefore there is ample opportunity to use hardware acceleration for further speedup over processors.

The exact effects of discretization and search horizon in HEV optimization are not fully known, but a reasonable starting point that is similar to related research [85, 86] is 30 steps for velocity and state of charge states, 30 steps for the combustion engine and electric motor torque commands, and 6 gears. With a search horizon of 512 steps. This means that one optimization cycle requires

$$30^2 \times 30^2 \times 6 \times 512 \approx 2.5 \text{ billion evaluations}, \tag{6}$$

making its use in real-time infeasible on processors. The technique has been used successfully on non-hybrid vehicles since they only need to keep track of velocity and simulate a combustion engine [87].

Hardware acceleration typically relies on some parallelism being exploitable in the problem, and this is no exception. The key observation here is that since the state reached by an input is always a state in a different time step, every state and, every input in those states can be evaluated in parallel since there are no data dependencies.

There is plenty of previous research into accelerators for dynamic programming for genome sequencing [88, 89, 90, 91], however, the state space and order in which states are visited is significantly different from the HEV case which makes them unsuitable for this application. Similarly, there is some previous research into running DP for XML *twig matching* [92] which also has a significantly different access pattern.

A closely related topic is Model Predictive Control (MPC) which also finds an optimal sequence of inputs that minimize some cost function. This is often implemented with *explicit* model predictive control where control laws are computed off-line leaving the on-line implementation to simply use lookup tables [93]. Since the on-line implementation only requires a lookup table it can be implemented very efficiently, but it relies on the state space not changing which rules out its use for HEV where the allowed state space varies based on road conditions, for example, speed limits. Explicit model predictive control on FPGAs is well researched, e.g. [94] and [95]. If the system model is linear which is not the case here, *linear* model predictive control can also be used and existing implementations of this on FPGAs have been surveyed in [96].

As far as the author is aware, there are no existing FPGA implementations of DP for optimal control of HEVs. There is, however, an implementation of a very similar algorithm to what is used here for GPUs [97].

6.5 Contributions

Traditionally, dynamic programming for optimal control of hybrid electric vehicles has only been feasible for offline use when executed on processors. In Paper E, we showed that it is possible in principle to run the algorithms in real-time using FPGAs, which has the potential to allow taking real-time data such as traffic conditions into account. This was followed up by Paper G in which we demonstrate

a scalable architecture for running the algorithm with a configurable number of parallel execution units. The speedup of this architecture over CPUs is significant: with four execution units, it finishes in 1.5% of the time used on a CPU.

Hardware to execute dynamic programming for optimal control has two major components. One component is the block which computes the simulation model, i.e. computes the next state and cost of inputs. The other component, the "executor" runs the dynamic programming algorithm by feeding the model with the appropriate inputs and keeping track of the outputs. While the two come together to form one system, the developer requirements between them are quite different. The executor requires careful management of memory resources to feed inputs and outputs to the model and requires precise control of where and when model evaluations are scheduled in order to achieve high performance. It is also a relatively generic block that only requires major changes if the algorithm itself is changed. For these reasons, developing the executor is a task best left to engineers with hardware development experience.

The model on the other hand is highly application-dependent, requires detailed knowledge of the system being modeled, and benefits from fine-tuning to maximize the output quality. For these reasons, an engineer with modeling experience is necessary for good results. Since it is a pure function from input to output, it can be efficiently implemented as a pipeline without feedback which makes it an ideal target for HLS.

For these reasons, in Paper G we argue that DP for optimal control on FPGAs should be implemented by a team of both hardware engineers and domain experts working in parallel with two tools separate tools. Domain experts can iterate on the model implementation with HLS tool without concerning themselves with any hardware details, while the hardware engineer works in parallel to develop the surrounding infrastructure in a lower-level language.

Chapter 7
Conclusions and Future Work

This dissertation revolves around three areas of the hardware design process. The Spade language addresses the need for improved productivity without sacrificing performance in traditional hardware description languages. A key principle in its design is that there is a lot to learn from modern software language design which has evolved much faster than the HDLs. Not all software features will work in hardware, which means that a big part of the design work is adapting existing abstractions like the type system to hardware or building new abstractions such as pipelining that is based on design principles used in software language design.

HDLs are a good fit for hardware engineers, but as accelerators are becoming more important, allowing domain experts to build hardware themselves has the potential for significant productivity improvements. Paper G proposes a workflow that allows vehicular systems experts to develop simulation models that can be used in conjunction with handwritten hardware for a well-performing system. In this area, a new HLS tool called Cinnabar was also developed. The focus of the tool is on synthesizing simulation models, especially those where bit-accuracy in the output is not a strict requirement. The combination of Cinnabar and the methodology to use it with Spade enables the use of dynamic programming to optimize hybrid electric vehicle fuel efficiency in real-time, which can lead to significant improvements in fuel efficiency and emissions.

Finally debugging is an unavoidable step in the hardware design process, and tools to ease debugging are essential. This is primarily addressed by Surfer, a waveform viewer that eases debugging of modern HDLs by offering an extensible translator system. Surfer is also built to be both extensible and embeddable which has resulted in it being used as a component in several commercial and open source projects.

7.1 Future work

All three projects have succeeded at their initial goals, and both Spade and Surfer have far exceeded the author's expectations. Spade, which started out as a fun side project, has grown into a full-fledged language. While it has not yet seen industry adoption, it is being used by several hobbyists, many of them with a software background that are now starting to explore the wonderful world of digital hardware. Surfer, which was created to address what the author thought

was a niche problem, has become the go-to waveform viewer for many hardware developers, is being used as a teaching tool at several universities, and is being used as a component of several commercial and open source projects.

Of course, languages and tools are never truly done, so there is plenty of future work to explore, both new avenues for research, additional engineering work, and community building.

7.1.1 Spade

For a long time, the Spade language has worked well for many designs, but there have been showstoppers preventing it from being used in some domains, in particular for very generic code. Four and a half years into the project though, most of those showstoppers are resolved which means more effort can be spent on refining the language to make it even more productive.

Native reasoning about clock domains is a logical next step as it categorically prevents accidental clock domain crossings, and lays the groundwork for improved handling of dynamic pipelines. The current pipelining system only has rudimentary support for stalls since there is no built-in way to communicate stall conditions to sub-pipelines. This means that pipelines play less well with other language features than they could. Modeling each unique stalled stage as an enable-condition for a clock domain may help resolve this issue.

Description of state machines is another area with ample opportunity for improvements. Currently, they are described manually using sum types and match expressions, but in practice, this is often as a very mechanical translation from a software model of loops and conditional control flow that the developer has in the back of their mind. One possible approach is YieldFSM [37], which implements a coroutine-based abstraction that interoperates well with the underlying RTL description in Clash on which it is based. This is an attractive approach since interoperability with the rest of the language is important, something that other higher level abstractions do not necessarily allow.

The clock gating optimization discussed in Paper B also shows that there is a potential for a high-level HDL to perform optimizations using the knowledge the compiler has about the semantics of the language. Further exploring this is an obvious next step to improving developer productivity and output quality.

For a language to be truly useful, it is not enough for the language design to be good, the ecosystem around the language also plays a key part. Perhaps most importantly, the language must have complete documentation for new and existing users. Some documentation does exist, but more needs to be written for it to not be a hindrance to further language adoption. Modern languages also often come with documentation generation tools for documenting libraries, this too is something that must be developed further than the current prototype. The build system works well, but the current dependency management has in many ways outgrown its shoes as more of the libraries in Spade start having transitive dependencies. The editor integration through the language server is in a similar spot, the current implementation works, but is far from its full potential.

While this list is long, the language has also grown a small community around it with some amazing people who are not only interested in playing with the

language but also actively improving it. This community is actively working on some of the tools discussed and has provided invaluable input on many of the recent language design decisions. Future work will certainly involve continuing to cultivate and grow the community.

7.1.2 Surfer

The Surfer project was started out of a concrete need for a way to translate bit representations of Spade values back into human-readable form, but it quickly grew into much more than that and is now an important part of the open-source EDA ecosystem. Like most tools, the list of potential improvements and future work is nearly infinite. The author's personal priority is still on the translator system that started the project. A big issue with the current implementation is that while it is possible to add new translators, doing so requires changing the source code of the tool itself and recompiling a custom version. A fully fledged WebAssembly based translator system is an appealing solution to this problem. The tool was originally written with little care for performance which is now becoming apparent when working with large files. Addressing these low-hanging performance improvements is also an obvious next step.

Even more so than Spade, Surfer has grown a healthy community of both users and contributors. Working with that community to make the tool as good as it can be is going to be more prioritized than specific features that the author himself is interested in.

7.1.3 Cinnabar and Hybrid Electric Vehicles

With both Spade and Surfer being more successful than anticipated, the list of future work for the second track of the dissertation is quite long, since there are plenty of unexplored research ideas.

Chief among them is further improving the fractional word length selection in Cinnabar. The current implementation uses one fractional word length across all signals, and that word length must be decided through simulation. There are likely some savings one could make by tuning fractional word length on a per-variable basis instead, but because a change in word length early in a computation has a compound effect on the output, doing so is not trivial. This may be possible to solve as an optimization problem where the word length is selected such that the total area of the design is minimized as long as the output error is within some user-bound bound. However, because word lengths are integers and the objective function of the output error is highly non-linear, this is likely to be a very difficult problem to solve.

The Cinnabar tool was built under the assumption that an HLS tool focused on simulation models could outperform a general-purpose tool, but whether or not this is true was not fully explored. Unlike the author's initial assumptions, commercial HLS tools appear to handle deeply pipelined hardware quite well, which makes the case for a dedicated tool for this application a tougher sell. However, while HLS tools have to retain bit accurate output, Cinnabar does not, which is likely to lead to some performance advantages, especially with the aforementioned improved

word length optimizations. Exploring how well existing HLS tools work in practice for this application is also highly interesting.

Bibliography

[1] J. Hennessy and D. Patterson, "A new golden age for computer architecture: Domain-specific hardware/software co-design, enhanced security, open instruction sets, and agile chip development," in *ACM/IEEE Int. Symp. Computer Architecture (ISCA)*, 2018, pp. 27–29. doi: 10.1109/ISCA.2018.00011.

[2] J. L. Hennessy and D. A. Patterson, "A new golden age for computer architecture," *Commun. ACM*, vol. 62, no. 2, pp. 48–60, Jan. 2019, doi: 10.1145/3282307.

[3] L. Bergstrom, "Beyond safety and speed: How Rust fuels team productivity." [Online]. Available: https://www.youtube.com/watch?v=QrrH2lcl9ew

[4] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek, "Benefits and drawbacks of adopting a secure programming language: Rust as a case study," in *Symp. Usable Privacy Security (SOUPS)*, USENIX Association, Aug. 2021, pp. 597–616.

[5] Z. Gao, C. Bird, and E. T. Barr, "To type or not to type: Quantifying detectable bugs in JavaScript," in *IEEE/ACM Int. Conf. Software Engineering (ICSE)*, IEEE, May 2017. doi: 10.1109/icse.2017.75.

[6] F. Khan, B. Chen, D. Varro, and S. McIntosh, "An empirical study of type-related defects in Python projects," *IEEE Trans. Softw. Eng.*, vol. 48, no. 8, pp. 3145–3158, Aug. 2022, doi: 10.1109/tse.2021.3082068.

[7] J. Tracey and I. Goldberg, "Grading on a curve: How Rust can facilitate new contributors while decreasing vulnerabilities," in *IEEE Secure Development Conf. (SecDev)*, IEEE, Oct. 2023. doi: 10.1109/secdev56634.2023.00016.

[8] WithoutBoats, "Ownership." [Online]. Available: https://without.boats/blog/ownership/

[9] H. Foster, "Part 6: The 2022 Wilson research group functional verification study." [Online]. Available: https://blogs.sw.siemens.com/verificationhorizons/2022/11/21/part-6-the-2022-wilson-research-group-functional-verification-study/

[10] Stack Overflow, "2023 developer survey." [Online]. Available: https://survey.stackoverflow.co/2023/

[11] "C++ compiler support." [Online]. Available: https://en.cppreference.com/w/cpp/compiler_support

[12] L. Klemmer and D. Große, "WAVING goodbye to manual waveform analysis in HDL design with WAL," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 10, pp. 3198–3211, Oct. 2024, doi: 10.1109/tcad.2024.3387312.

[13] E. D. Sozzo, D. Conficconi, A. Zeni, M. Salaris, D. Sciuto, and M. D. Santambrogio, "Pushing the level of abstraction of digital system design: A survey on how to program FPGAs," *ACM Comput. Surveys*, vol. 55, no. 5, pp. 1–48, Dec. 2022, doi: 10.1145/3532989.

[14] R. Nigam, P. H. Azevedo de Amorim, and A. Sampson, "Modular hardware design with timeline types," *Proc. ACM Program. Lang. (PLDI)*, vol. 7, pp. 343–367, Jun. 2023, doi: 10.1145/3591234.

[15] L. Van Hirtum and C. Plessl, "Latency counting in the SUS language," in *Workshop Lang. Tools Tech. Accelerator Design*, 2024.

[16] J. Pu *et al.*, "Programming heterogeneous systems from an image processing DSL," *ACM Trans. Architecture Code Optimization*, vol. 14, no. 3, pp. 1–25, Aug. 2017, doi: 10.1145/3107953.

[17] Y. E. Oron Port, "Registerless hardware description," in *Workshop Lang. Tools Tech. Accelerator Design*, 2021.

[18] B. Stroustrup, "Foundations of C++," *Programming Languages and Systems*. Springer, Berlin, Heidelberg, pp. 1–25, 2012. doi: 10.1007/978-3-642-28869-2_1.

[19] A. Turon, "Abstraction without overhead: Traits in Rust." [Online]. Available: https://blog.rust-lang.org/2015/05/11/traits/

[20] CIRCT Project, "CIRCT." [Online]. Available: https://circt.llvm.org/

[21] C. Kern and M. R. Greenstreet, "Formal verification in hardware design: A survey," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 4, no. 2, pp. 123–193, Apr. 1999, doi: 10.1145/307988.307989.

[22] A. Cohn, "The notion of proof in hardware verification," *J. Automated Reasoning*, vol. 5, no. 2, pp. 127–139, Jun. 1989, doi: 10.1007/bf00243000.

[23] A. Lööw, "The simulation semantics of synthesisable Verilog," *Proc. ACM Program. Lang. (PLDI)*, vol. 9, no. OOPSLA1, pp. 1295–1320, Apr. 2025, doi: 10.1145/3720484.

[24] L. Klemmer and D. Große, "Versatile hardware analysis techniques: From Waveform-based analysis to formal verification," 1st ed. Springer Nature Switzerland, 2025. doi: 10.1007/978-3-031-83093-8.

[25] J. Bachrach *et al.*, "Chisel: Constructing hardware in a Scala embedded language," in *Proc. Des. Automat. Conf.*, Jun. 2012, pp. 1212–1221. doi: 10.1145/2228360.2228584.

[26] SpinalHDL contributors, "SpinalHDL." [Online]. Available: https://github.com/SpinalHDL/SpinalHDL

[27] C. Baaij, "Digital circuits in ClaSH," University of Twente, 2015. doi: 10.3990/1.9789036538039.

[28] Amaranth contributors, "Amaranth HDL." [Online]. Available: https://github.com/amaranth-lang/amaranth

[29] R. Nikhil, "Bluespec SystemVerilog: Efficient, correct RTL from high-level specifications," in *Proc ACM/IEEE Int. Conf. Formal Methods Models Co-Design*, IEEE, 2004, pp. 69–70. doi: 10.1109/memcod.2004.1459818.

[30] J. Kemmerer, "PipelineC: Easy open-source hardware description between RTL and HLS," in *Proc. Workshop Open-Source EDA Technol.*, 2022.

[31] Veryl Contributors, “Veryl hardware description language.” [Online]. Available: https://veryl-lang.org/
[32] Redwood EDA, “TL-Verilog.” [Online]. Available: https://web.archive.org/web/20221006080731/https://www.redwoodeda.com/tl-verilog
[33] M. Korbel, “Rapid open hardware development framework,” in *Proc. Workshop Open-Source EDA Technol.*, 2022.
[34] O. Port and Y. Etsion, “DFiant: A dataflow hardware description language,” in *Proc. Int. Conf. Field Programmable Logic Appl.*, 2017. doi: 10.23919/fpl.2017.8056858.
[35] S. Lefebvre, “Silice.” [Online]. Available: https://github.com/sylefeb/Silice/tree/5003ec72
[36] A. Forster, “CoHDL.” [Online]. Available: https://github.com/alexander-forster/cohdl
[37] M. Materzok, “Generating circuits with generators,” *Proc. ACM Program. Lang. (PLDI)*, vol. 6, no. ICFP, pp. 52–79, Aug. 2022, doi: 10.1145/3549821.
[38] L.-y. Xia, “Where does the name "algebraic data type" come from?.” [Online]. Available: https://blog.poisson.chat/posts/2024-07-26-adt-history.html
[39] F. Skarman and O. Gustafsson, “Spade: An expression-based HDL with pipelines,” in *Workshop Open-Source Des. Automat.*, Apr. 2023.
[40] S. Basu, “Rust as a hardware description language,” in *Workshop Lang. Tools Tech. Accelerator Design*, 2024.
[41] D. Walker, “Substructural type systems,” *Advanced topics in types and programming languages*. MIT Press, Cambridge, Mass., pp. 3–43, 2002.
[42] R. Nigam *et al.*, “Predictable accelerator design with time-sensitive affine types,” in *Proc. ACM SIGPLAN Conf. Program. Language Des. Implementation*, ACM, Jun. 2020. doi: 10.1145/3385412.3385974.
[43] S. Han, M. Jang, and J. Kang, “ShakeFlow: Functional hardware description with latency-insensitive interface combinators,” in *Proc. ACM Int. Conf. Arch. Support Programming Lang. Operating Syst.*, in ASPLOS ’23, vol. 2. ACM, Jan. 2023, pp. 702–717. doi: 10.1145/3575693.3575701.
[44] R. Nigam, E. Gabizon, E. Lam, and A. Sampson, “Correct and compositional hardware generators.” arXiv, 2024. doi: 10.48550/ARXIV.2401.02570.
[45] kcamenzind, “Intro BlueSpec user guide.” [Online]. Available: https://github.com/kcamenzind/BluespecIntroGuide
[46] J. Choi, M. Vijayaraghavan, B. Sherman, A. Chlipala, and Arvind, “Kami: A platform for high-level parametric hardware specification and its modular verification,” *Proc. ACM Program. Lang. (PLDI)*, vol. 1, no. ICFP, pp. 1–30, Aug. 2017, doi: 10.1145/3110268.
[47] T. Bourgeat, C. Pit-Claudel, A. Chlipala, and Arvind, “The essence of Bluespec: A core language for rule-based hardware design,” in *Proc. ACM SIGPLAN Conf. Program. Language Des. Implementation*, in PLDI ’20. ACM, Jun. 2020, pp. 243–257. doi: 10.1145/3385412.3385965.
[48] Y. Xiao, Z. Luo, K. Zhou, and Y. Liang, “Cement: Streamlining FPGA hardware design with cycle-deterministic eHDL and synthesis,” in *Proc. ACM/SIGDA Int. Symp. Field Programmable Gate Arrays*, in FPGA ’24. ACM, Apr. 2024, pp. 211–222. doi: 10.1145/3626202.3637561.

[49] Y. Xiao, Z. Luo, and Y. Liang, “cmt2: Rule-based hardware description in Rust with temporal semantics,” in *Workshop Lang. Tools Tech. Accelerator Design*, 2025.
[50] “ReWire: Functional hardware description.” [Online]. Available: https://rewire-hardware.github.io/ReWire/
[51] T. Strauch, “Dynamic inside-out verification using inverse transactions in TLM,” in *Forum on Specification & Design Languages (FDL)*, IEEE, Sep. 2018, pp. 5–16. doi: 10.1109/fdl.2018.8524048.
[52] “Silver Oak.” [Online]. Available: https://github.com/project-oak/silveroak
[53] Z. Shuman *et al.*, “Utökning av Swim: – ett byggsystem för moderna hårdvarubeskrivande språk,” Linköping University, 2024.
[54] L. Klemmer and D. Große, “WSVA: a SystemVerilog assertion to WAL compiler,” in *Workshop Open-Source Des. Automat.*, 2024.
[55] R. Nane *et al.*, “A survey and evaluation of FPGA high-level synthesis tools,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 35, no. 10, pp. 1591–1604, Oct. 2016, doi: 10.1109/tcad.2015.2513673.
[56] C. Pilato and F. Ferrandi, “Bambu: A modular framework for the high level synthesis of memory-intensive applications,” in *Proc. Int. Conf. Field Programmable Logic Appl.*, IEEE, Sep. 2013. doi: 10.1109/fpl.2013.6645550.
[57] N. B. Agostini *et al.*, “SODA synthesizer: An open-source, multi-level, modular, extensible compiler from high-level frameworks to silicon,” in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design*, in ICCAD ’22. ACM, Oct. 2022, pp. 1–7. doi: 10.1145/3508352.3561101.
[58] C. Lattner *et al.*, “MLIR: A compiler infrastructure for the end of Moore's law.” arXiv, 2020. doi: 10.48550/ARXIV.2002.11054.
[59] A. Canis *et al.*, “LegUp: An open-source high-level synthesis tool for FPGA-based processor/accelerator systems,” *ACM Trans. Embedded Comput. Syst.*, vol. 13, no. 2, pp. 1–27, Sep. 2013, doi: 10.1145/2514740.
[60] Microchip, “SmartHLS™ compiler software.” [Online]. Available: https://www.microchip.com/en-us/products/fpgas-and-plds/fpga-and-soc-design-tools/smarthls-compiler
[61] L. Josipovic, P. Brisk, and P. Ienne, “From C to elastic circuits,” in *Proc. Asilomar Conf. Signals Syst. Comput.*, IEEE, Oct. 2017. doi: 10.1109/acssc.2017.8335150.
[62] L. Josipović, A. Guerrieri, and P. Ienne, “Invited tutorial: Dynamatic: From C/C++ to dynamically scheduled circuits,” in *Proc. ACM/SIGDA Int. Symp. Field Programmable Gate Arrays*, in FPGA ’20. ACM, Feb. 2020, pp. 1–10. doi: 10.1145/3373087.3375391.
[63] MathWorks, “HDL Coder.” [Online]. Available: https://www.mathworks.com/products/hdl-coder.html
[64] Google, “XLS: Accelerated HW synthesis.” [Online]. Available: https://google.github.io/xls
[65] Google, “DSLX reference.” [Online]. Available: https://google.github.io/xls/dslx_reference/
[66] U. Kumar and F. Rivoallon, “Opening a world of possibilities: Vitis HLS front-end is now open source for all on GitHub.” [Online]. Available: https://www.

xilinx.com/about/blogs/ai-and-machine-learning-blog/2021/vitis-hls-front-end-now-open-source.html
[67] F. Ferrandi *et al.*, "Invited: Bambu: An open-source research framework for the high-level synthesis of complex applications," in *Proc. Des. Automat. Conf.*, IEEE, Dec. 2021. doi: 10.1109/dac18074.2021.9586110.
[68] Q. Huang *et al.*, "The effect of compiler optimizations on high-level synthesis-generated hardware," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 8, no. 3, pp. 1–26, May 2015, doi: 10.1145/2629547.
[69] L. Josipovic, A. Guerrieri, and P. Ienne, "From C/C++ code to high-performance dataflow circuits," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 7, pp. 2142–2155, Jul. 2022, doi: 10.1109/tcad.2021.3105574.
[70] "Dynamatic." [Online]. Available: https://dynamatic.epfl.ch/
[71] J. Ragan-Kelley, A. Adams, S. Paris, M. Levoy, S. Amarasinghe, and F. Durand, "Decoupling algorithms from schedules for easy optimization of image processing pipelines," *ACM Trans. Graphics*, vol. 31, no. 4, pp. 1–12, Jul. 2012, doi: 10.1145/2185520.2185528.
[72] H. Chen, N. Zhang, S. Xiang, Z. Zeng, M. Dai, and Z. Zhang, "Allo: A programming model for composable accelerator design," *Proc. ACM Program. Lang. (PLDI)*, vol. 8, no. PLDI, pp. 593–620, Jun. 2024, doi: 10.1145/3656401.
[73] D. Koeplinger *et al.*, "Spatial: A language and compiler for application accelerators," in *Proc. ACM SIGPLAN Conf. Program. Language Des. Implementation*, in PLDI '18. ACM, Jun. 2018, pp. 296–311. doi: 10.1145/3192366.3192379.
[74] GTKWave contributors, "GTKWave." [Online]. Available: https://gtkwave.sourceforge.net/
[75] A. Chapyzhenka and J. Probell, "WaveDrom rendering beautiful waveforms from plain text," in *Proc. Synopsys User Group*, 2016.
[76] R. Meloni, H. P. Hofstee, and Z. Al-Ars, "Tywaves: A typed waveform viewer for Chisel," in *Proc. Nordic Circuits Syst. Conf.*, IEEE, Oct. 2024, pp. 1–6. doi: 10.1109/norcas64408.2024.10752465.
[77] K. V. Singh, H. O. Bansal, and D. Singh, "A comprehensive review on hybrid electric vehicles: Architectures and components," *J. Modern Transportation*, vol. 27, no. 2, pp. 77–107, Mar. 2019, doi: 10.1007/s40534-019-0184-3.
[78] A. A. Pesaran, T. Markel, H. S. Tataria, and D. Howell, "Battery requirements for plug-in hybrid electric vehicles – analysis and rationale," National Renewable Energy Laboratory, 2007.
[79] A. D. Lewis, "The maximum principle of Pontryagin in control and in optimal control," Department of Mathematics, Statistics, Queen's University,Kingston, May 2006.
[80] M. I. Kamien and N. L. Schwartz, "Dynamic optimization," 4. pr., no. 4. in Dynamic economics. New York [u.a.]: North-Holland, 1985.
[81] Y. Quing, "Introduction to the optimal control theory and some applications." [Online]. Available: https://math.uchicago.edu/~may/REU2019/REUPapers/Qing.pdf
[82] L. V. Pérez, G. R. Bossio, D. Moitre, and G. O. García, "Optimization of power management in an hybrid electric vehicle using dynamic program-

ming," *Math. Comput. Simul.*, vol. 73, no. 1–4, pp. 244–254, Nov. 2006, doi: 10.1016/j.matcom.2006.06.016.

[83] R. Bellman, "The theory of dynamic programming," *Bull. Amer. Math. Soc.*, vol. 60, no. 6, pp. 503–516, Nov. 1954, doi: 10.1090/s0002-9904-1954-09848-8.

[84] D. P. Bertsekas, "Dynamic programming and optimal control." Athena Scientific, 1995.

[85] S. R. Deshpande, S. Gupta, A. Gupta, and M. Canova, "Real-time ecodriving control in electrified connected and autonomous vehicles using approximate dynamic programing," *J. Dynamic Syst. Meas. Control*, vol. 144, no. 1, Jan. 2022, doi: 10.1115/1.4053292.

[86] S. R. Deshpande, D. Jung, L. Bauer, and M. Canova, "Integrated approximate dynamic programming and equivalent consumption minimization strategy for eco-driving in a connected and automated vehicle," *IEEE Trans. Veh. Technol.*, vol. 70, no. 11, pp. 11204–11215, Nov. 2021, doi: 10.1109/tvt.2021.3102505.

[87] E. Hellström, M. Ivarsson, J. Åslund, and L. Nielsen, "Look-ahead control for heavy trucks to minimize trip time and fuel consumption," *Control Eng. Pract.*, vol. 17, no. 2, pp. 245–254, Feb. 2009, doi: 10.1016/j.conengprac.2008.07.005.

[88] S. O. Settle, "High-performance dynamic programming on FPGAs with OpenCL," in *Proc. IEEE High Perform. Extreme Comput. Conf.*, 2013, pp. 1–6.

[89] Y. Hu and P. Georgiou, "A study of the partitioned dynamic programming algorithm for genome comparison in FPGA," in *Proc. IEEE Int. Symp. Circuits Syst.*, May 2013, pp. 1897–1900. doi: 10.1109/ISCAS.2013.6572237.

[90] L. Di Tucci, K. O'Brien, M. Blott, and M. D. Santambrogio, "Architectural optimizations for high performance and energy efficient Smith-Waterman implementation on FPGAs using OpenCL," in *Proc. Des. Automat. Test in Eur. Conf.*, IEEE, Mar. 2017, pp. 716–721. doi: 10.23919/date.2017.7927082.

[91] S. Natarajan, N. KrishnaKumar, M. Pavan, D. Pal, and S. K. Nandy, "ReneGENE-DP: Accelerated parallel dynamic programming for genome informatics," in *Proc. IEEE Int. Conf. Electronics, Comput. Commun. Technol.*, IEEE, Mar. 2018. doi: 10.1109/conecct.2018.8482378.

[92] R. Moussalli, M. Salloum, W. Najjar, and V. J. Tsotras, "Massively parallel XML twig filtering using dynamic programming on FPGAs," in *Proc. IEEE Int. Conf. Data Eng.*, IEEE, Apr. 2011. doi: 10.1109/icde.2011.5767899.

[93] A. Bemporad, "Explicit model predictive control," *Encyclopedia of Systems and Control*. Springer London, pp. 1–9, 2013. doi: 10.1007/978-1-4471-5102-9_10-1.

[94] D. Ingole, J. Holaza, B. Takacs, and M. Kvasnica, "FPGA-based explicit model predictive control for closed-loop control of intravenous anesthesia," in *Proc. Int. Conf. Process Control*, IEEE, Jun. 2015. doi: 10.1109/pc.2015.7169936.

[95] A. Gersnoviez, M. Brox, and I. Baturone, "High-speed and low-cost implementation of explicit model predictive controllers," *IEEE Trans. Control*

Syst. Technol., vol. 27, no. 2, pp. 647–662, Mar. 2019, doi: 10.1109/tcst.2017.2775187.

[96] I. McInerney, G. A. Constantinides, and E. C. Kerrigan, "A survey of the implementation of linear model predictive control on FPGAs," *IFAC-PapersOnLine*, vol. 51, no. 20, pp. 381–387, 2018, doi: 10.1016/j.ifacol.2018.11.063.

[97] Z. Zhu, S. Gupta, N. Pivaro, S. R. Deshpande, and M. Canova, "A GPU implementation of a look-ahead optimal controller for eco-driving based on dynamic programming," in *Proc. Europ. Control Conf.*, IEEE, Jun. 2021. doi: 10.23919/ecc54610.2021.9655197.

Part 2
Publications

Papers

The papers associated with this thesis have been removed for copyright reasons. For more details about these see:

https://doi.org/10.3384/9789181181777

PhD Dissertations
Division of Electronics and Computer Engineering
Linköping University

C. Bae Implementation and Design of High-Speed FIR Filters for Chromatic Dispersion Compensation. No. 2431, 2025. ISBN: 978-91-8075-976-2.

N.M. Sarband Energy-Efficient Implementation of Communication Algorithms through Complexity Reduction. No. 2414, 2025. ISBN: 978-91-8075-851-2.

Y.H. Kifle Design of Near-Field Wireless-Powered Biphase Implantable Stimulators. No. 2237, 2022. ISBN: 978-91-7929-371-0.

O.A. Morales Chacón Studies on the Performance Bounds and Design of Current-Steering DACs. No. 2238, 2022. ISBN: 978-91-7929-373-4.

R. Sadeghifar Studies on Selected Topics in Radio Frequency Digital-to-Analog Converters. No. 1999, 2019. ISBN: 978-91-7685-030-5.

M.T. Pasha All-Digital PWM Transmitters. No. 1972, 2019.
ISBN: 978-91-7685-153-1.

M.F. Ul Haque Pulse-Width Modulated RF Transmitters. No. 1822, 2017.
ISBN: 978-91-7685-598-0.

K. Chen Energy-Efficient Data Converters for Low-Power Sensors. No. 1816, 2017. ISBN: 978-91-7685-617-8.

V. Unnikrishnan Design of VCO-based ADCs. No. 1812, 2016 .
ISBN: 978-91-7685-624-6.

A. Truskog Design of Energy-Efficient High-Performance ASIP-DSP Platforms. No. 1772, 2016. ISBN: 978-91-7685-745-8.

Q.T. Duong Efficient Integrated Circuits for Wideband Wireless Transceivers. No. 1722, 2016. ISBN: 978-91-7685-904-9.

S.A. Alam Techniques for Efficient Implementation of FIR and Particle Filtering. No. 1716, 2016. ISBN: 978-91-7685-915-5.

P. Harikumar Low-Voltage Analog-to-Digital Converters and Mixed-Signal Interfaces. No. 1728, 2016. ISBN: 978-91-7685-890-5.

I. Kazim Variation-Aware System Design Simulation Metodology for Capacitive BCC Transceivers. No. 1721, 2015. ISBN: 978-91-7685-906-3.

A. Bhide Desing of High-Speed Time-Interleaved Delta-Sigma D/A Converters. No. 1688, 2015. ISBN: 978-91-7519-017-4.

J. Sohl Efficient Compilation for Application Specific Instruction set Processors with Multi-bank Memories. No. 1641, 2015. ISBN: 978-91-7519-151-5.

N. Afzal Complexity and Power Reduction in Digital Delta-Sigma Modulators. No. 1640, 2015. ISBN: 978-91-7519-154-6.

N. Andersson Design of Integrated Building Blocks for the Digital/Analog Interface. No. 1638, 2015. ISBN: 978-91-7519-163-8.

F. Qazi Selected Applications of Switched Capacitor Circuits: RF N-Path Filters and Modulators. No. 1627, 2015. ISBN: 978-91-7519-210-9.

A. Ojani Analysis and Design of DLL-Based Frequency Synthesizers for Ultra-Wideband Communication. No. 1618, 2014. ISBN: 978-91-7519-248-2.

D. Zhang Ultra-Low-Power Analog-to-Digital Converters for Medical Applications. No. 1611, 2014. ISBN: 978-91-7519-264-2.

J. Wang Low Overhead Memory Subsystem Design for a Multicore Parallel DSP Processor. No. 1532, 2014. ISBN: 978-91-7519-556-8.

A. Fazli Yeknami Low-Power Delta-Sigma Modulators for Medical Applications. No. 1563, 2014. ISBN: 978-91-7519-446-2.

F. Qureshi Optimization of Rotations in FFTs. No. 1423, 2012. ISBN: 978-91-7519-973-3.

M. Abbas On the Implementation of Integer and Non-Integer Sampling Rate Conversion. No. 1420, 2012. ISBN: 978-91-7519-980-1.

Z.U. Sheikh Efficient Realizations of Wide-Band and Reconfigurable FIR Systems. No. 1424, 2012. ISBN: 978-91-7519-972-6.

J. Fritzin CMOS RF Power Amplifiers for Wireless Communications. No. 1399, 2011. ISBN: 978-91-7393-059-8.

A. Blad Low Complexity Techniques for Low Density Parity Check Code Decoders and Parallel Sigma-Delta ADC Structures. No. 1385, 2011. ISBN: 978-91-7393-104-5.

T. Sundström Design of High-Speed Analog-to-Digital Converters using Low-Accuracy Components. No. 1367, 2011. ISBN: 978-91-7393-203-5.

A. Eghbali Contributions to Reconfigurable Filter Banks and Transmultiplexers. No. 1344, 2010. ISBN: 978-91-7393-296-7.

P.A. Karlström NoGAP: Novel Generator of Accelerators and Processors. No. 1347, 2010. ISBN: 978-91-7393-293-6.

S. Ahmad Stimuli Generation Techniques for On-Chip Mixed-Signal Test. No. 1350, 2010. ISBN: 978-91-7393-288-2.

R. Asghar Flexible Interleaving Sub-systems for FEC in Baseband Processors. No. 1312, 2010. ISBN: 978-91-7393-397-1.

D. Wu Scalable Multi-Standard Radio Baseband for Modern Wireless Communication. No. 1279, 2009. ISBN: 978-91-7393-507-4.

J. Eilert ASIP for Wireless Communication and Media. No. 1298, 2010. ISBN: 978-91-7393-450-3.

N. Ahsan Reconfigurable and Broadband Circuits for Flexible RF Front Ends. No. 1259, 2009. ISBN: 978-91-7393-605-7.

M.R. Ramzan Flexible Wireless Receivers: On-Chip Testing Techniques and Design for Testability. No. 1261, 2009. ISBN: 978-91-7393-601-9.

A. Ehliar Performance Driven FPGA Design with an ASIC Perspective. No. 1237, 2009. ISBN: 978-91-7393-702-3.

B. Mesgar Zadeh Low-Power Low-Jitter Clock Generation and Distribution. No. 1208, 2008. ISBN: 978-91-7393-817-4.

K. Johansson Low Power and Low Complexity Shift-and-Add Based Computations. No. 1201, 2008. ISBN: 978-91-7393-836-5.

M. Hansson Low-Power Clocking and Circuit Techniques for Leakage and Process Variation Compensation. No. 1197, 2008. ISBN: 978-91-7393-847-1.

M. Olsson Contributions to Delay Gain and Offset Estimation. No. 1189, 2008. ISBN: 978-91-7393-883-9.

H. Fredriksson Improvement Potential and Equalization Circuit Solutions for Multi-drop DRAM Memory Buses. No. 1177, 2008. ISBN: 978-91-7393-910-2.

S. Vangal Performance and Energy Efficient Network-on-Chip Architectures. No. 1130, 2007. ISBN: 978-91-85895-91-5.

A.H. Nilsson Design of Programmable Multi-Standard Baseband Processors. No. 1084, 2007. ISBN: 978-91-85715-44-2.

L. Rosenbaum On low-complexity frequency selective digital filters and filter banks. No. 1097, 2007. ISBN: 978-91-85715-07-7.

E. Säll Implementation of Flash Analog-do-Digital Converters in Silicon-on-Insulator CMOS Technology. No. 1093, 2007. ISBN: 978-91-85715-18-3.

E. Backenius Reduction of Substrate Noise in Mixed-Signal Circuits. No. 1094, 2007. ISBN: 978-91-85715-12-1.

J. Carlsson Contributions to Asynchronous Communication Ports for GALS Systems. No. 1062, 2006. ISBN: 91-85643-30-0.

S. Andersson Multiband LNA Design and RF-Sampling Front-Ends for Flexible Wireless Receivers. No. 1036, 2006. ISBN: 91-85523-22-4.

R. Hägglund An Optimization-Based Approach to Efficient Design of Analog Circuits. No. 1026, 2006. ISBN: 91-85523-61-5.

E. Hjalmarson A Computer-Aided Approach to Design of Robust Analog Circuits. No. 1025, 2006. ISBN: 91-85523-62-3.

P. Caputa Efficient High-Speed On-Chip Global Interconnects. No. 992, 2006. ISBN: 91-85457-87-6.

E. Tell Design of Programmable Baseband Processors. No. 969, 2005. ISBN: 91-85457-20-5.

M. Karlsson Implementation of Digit-Serial Filters. No. 952, 2005. ISBN: 91-85299-57-X.

O. Andersson Modeling and Implementation of Current-Steering Digital-to Analog Converters. No. 944, 2005. ISBN: 91-8529-796-8.

H. Olsson Studies on Design and Implementation of Low-Complexity Digital Filters. No. 949, 2005. ISBN: 91-85299-42-1.

D. Eidenskog Development and Performance Evaluation of Networks on Chip. No. 932, 2005. ISBN: 91-85297-62-3.

K.G. Folkesson ADC Modeling from a System Perspective and Design o RF-Sampling Radio Receivers. No. 911, 2004. ISBN: 91-8529-588-4.

H. Bengtsson High Speed CMOS Optical Receiver. No. 904, 2004. ISBN: 91-85295-61-2.

D. Jakonis Direct RF Sampling Receivers for Wireless Systems in CMOS Technology. No. 881, 2004. ISBN: 91-7373-965-0.

U. Nordqvist Protocol Processing in Network Terminals. No. 865, 2004. ISBN: 91-7373-914-6.

O. Gustafsson Contributions to Low-Complexity Digital Filters. No. 837, 2003. ISBN: 91-7373-702-X.

T. Henriksson Intra-Packet Data-Flow Protocol Processor. No. 813, 2003. ISBN: 91-7373-624-4.

A. Rantzer Photo Diodes for Machine Vision. No. 799, 2003. ISBN: 91-7373-602-3.

P. Löwenborg Asymmetric Filter Banks for Mitigation of Mismatch Errors in High-Speed Analog-to-Digital Converters. No. 787, 2002. ISBN: 91-7373-471-3.

M. Duppils Digitally Controlled Analog Multiply-Accumulate Units. No. 792, 2002. ISBN: 91-7373-478-0.

I. Söderquist CMOS Circuits for Digital RF Systems. No. 775, 2002. ISBN: 91-7373-429-2.

M. Rudberg DSP Algorithms and Architectures for Telecommunication. No. 705, 2001. ISBN: 91-737-3069-6.

J. Wikner Studies on CMOS Digital-to Analog Converters. No. 667, 2001. ISBN: 91-721-9910-5.

H. Turbell Cone-Beam Reconstruction using Filtered Backprojection. No. 672, 2001. ISBN: 91-7219-919-9.

FACULTY OF SCIENCE AND ENGINEERING

Linköping Studies in Science and Technology, Dissertation No. 2460, 2025
Department of Electrical Engineering

Linköping University
SE-581 83 Linköping, Sweden

www.liu.se

